

SMALL-CAP STOCK TRADING STRATEGIES FOR RETAIL TRADERS

A Data-Driven, Statistically Validated, and Reproducible Study of Strategy Design, Risk Management, and Technical-Pattern-Based Signals

How Can Retail Traders Trading US Markets Design, Test, and Deploy Profitable Small-Cap Stock Trading Strategies? A Data-Driven, Statistically Validated, and Reproducible Study of Strategy Design, Risk Management, and Technical-Pattern-Based Signals

Bachelor of Science in Computer Science

Sandip Poudel

Metropolitan State University, St. Paul, MN, USA

December 2025

ABSTRACT

Small-cap equities (market capitalization \$300 million to \$2 billion) present unique opportunities and challenges for retail traders due to their high volatility, liquidity constraints, and information asymmetries. This dissertation addresses five core research questions spanning strategy design, validation, risk management, automation, and practical deployment constraints.

We develop and rigorously validate six distinct strategy families: (1) volatility-scaled momentum with liquidity filters, (2) gap-and-go opening range breakout variants, (3) mean reversion after extreme intraday moves, (4) breakout-retest pattern recognition, (5) event-driven volatility strategies with timestamp discipline, and (6) market regime-filtered composite strategies. Each strategy is specified with mathematical rigor, implemented in vectorized and event-driven Python code, and validated using walk-forward analysis with nested cross-validation, point-in-time data handling, and multiple-hypothesis testing corrections.

This work demonstrates that small-cap trading strategies can achieve risk-adjusted returns exceeding 0.8 Sharpe ratio in out-of-sample periods when properly designed with realistic transaction costs (\$0.01–\$0.05 spreads), commissions (\$0–\$1.00 per share), and liquidity constraints (minimum average volume \$500K daily). Critical contributions include: (1) a reproducible bias-control framework preventing survivorship bias, look-ahead bias, and data-snooping bias; (2) stress tests across market regimes (normal, crisis, regime shifts); (3) operational safeguards for retail broker APIs (Alpaca, Interactive Brokers, Tradier) including order reconciliation and kill-switch logic; and (4) an ethics framework addressing pump-and-dump risk and market manipulation.

This dissertation is intended for graduate-level committees in Computer Science, quantitative finance practitioners, and retail traders seeking statistically rigorous, reproducible implementations of small-cap trading strategies.

Keywords: small-cap trading, momentum, mean reversion, pattern recognition, walk-forward validation, retail trading algorithms, execution risk, risk management, technical analysis, quantitative finance.

TABLE OF CONTENTS

1. Introduction
2. Literature Review
3. Data, Universe Construction, and Bias Control
4. Methodology & Statistical Validation
5. Strategy Families: Detailed Specifications
6. Risk Management Framework
7. Results and Backtesting Analysis
8. Implementation & Automation
9. Discussion, Limitations, and Future Work
10. Appendices

CHAPTER 1: INTRODUCTION

1.1 Motivation and Problem Statement

The retail trading landscape has evolved dramatically over the past decade. Commission-free brokers (Alpaca, TradeStation, Tradier), accessible APIs, and improved data availability have democratized access to automated trading. However, small-cap equities—historically the domain of professional traders and hedge funds—remain underexplored in academic literature on automated retail trading.

Small-cap stocks are defined operationally in this work as US-listed equities with market capitalization between \$300 million and \$2 billion and average daily volume of at least \$500,000. These securities exhibit characteristics distinctly different from large-cap stocks:

- **Volatility:** Intraday moves of 5–15% are common; daily gaps of 10–20% are frequent.
- **Spread:** Bid-ask spreads of \$0.02–\$0.10 are typical (vs. \$0.01 for liquid blue chips).
- **Volume concentration:** Often driven by a small number of market participants; vulnerable to liquidity shocks.
- **Information asymmetry:** Fewer sell-side analysts; less institutional interest; more retail-driven price discovery.
- **Gaps and halts:** Gap up/down openings occur frequently; trading halts for news dissemination are common.
- **Borrow availability:** Short-selling small caps is constrained by limited borrow supply and elevated locate costs.

These characteristics present both opportunities and hazards. The *opportunity* is that small-cap volatility and lower information efficiency create pattern-based trading opportunities missed by liquid-only strategies. The *hazard* is that retail traders without rigorous risk management and execution discipline will suffer severe losses.

1.2 Research Objectives and Contributions

This dissertation addresses the following research questions:

RQ1: Which classes of small-cap strategies are plausible for retail traders given realistic liquidity, spread, volatility, and short-availability constraints?

RQ2: Which signal families (momentum, mean reversion, pattern recognition, event-driven, microstructure-aware) survive robust out-of-sample validation and multiple-hypothesis testing correction?

RQ3: What risk management and position-sizing approaches best control drawdowns and tail risk for small caps?

RQ4: How sensitive are strategy results to transaction costs, market regimes, and liquidity filters?

RQ5: How can strategies be automated responsibly through retail broker APIs with appropriate safeguards?

1.3 Main Contributions

Methodological contributions:

1. A rigorous, reproducible bias-control framework preventing survivorship bias (point-in-time delisting handling), look-ahead bias (correct date alignment), and data-snooping bias (walk-forward + nested cross-validation + FDR correction).
2. A comprehensive small-cap universe construction methodology with explicit liquidity, price, and market-cap filters; corporate action handling; and “as-of” join procedures.
3. Detailed specification of six strategy families with mathematical precision, pseudocode, and computable feature engineering.

Empirical contributions:

4. Out-of-sample validation showing which strategies survive realistic transaction costs and market regimes.
5. Sensitivity analysis demonstrating performance degradation as spreads widen, volume declines, or market regime shifts.
6. Stress tests on the 2020 COVID crash, 2022 interest-rate shock, and 2024 market volatility events.

Software and operational contributions:

7. A complete, modular Python backtesting and live-trading framework (data layer, feature layer, signal layer, portfolio layer, execution layer, reporting layer).
8. Integration examples with three major retail brokers (Alpaca, Interactive Brokers, Tradier), including order reconciliation, error handling, and kill-switch logic.
9. Unit tests, integration tests, and operational monitoring examples (logging, alerting, position reconciliation).

1.4 Dissertation Structure

This dissertation proceeds as follows:

- **Chapter 2** reviews relevant literature: momentum and mean-reversion effects, small-cap microstructure, backtesting pitfalls, and regulatory considerations.
- **Chapter 3** specifies the data layer: universe construction, data sources, bias-control mechanisms, and corporate action handling.
- **Chapter 4** details the validation methodology: walk-forward analysis, nested cross-validation, multiple-hypothesis testing correction, and risk metric definitions.
- **Chapter 5** provides detailed specifications of six strategy families with mathematical formalism and pseudocode.
- **Chapter 6** develops the risk management framework: position sizing, portfolio constraints, stop policies, and disaster controls.

- **Chapter 7** presents backtesting results: performance tables, ablation studies, sensitivity analysis, and regime stress tests.
- **Chapter 8** covers implementation: Python code architecture, broker integration, execution safeguards, and monitoring.
- **Chapter 9** discusses limitations, ethical considerations, and directions for future work.
- **Appendices** include full code listings, mathematical derivations, and supplementary tables.

CHAPTER 2: LITERATURE REVIEW

2.1 Academic Foundation: Momentum and Mean Reversion

The academic literature on momentum and mean reversion spans decades. Jegadeesh and Titman (1993) demonstrated that stocks with strong prior performance (past 3–12 months) tend to outperform over the subsequent 3–12 months—a phenomenon termed *momentum*. Conversely, De Bondt and Thaler (1985) showed that extreme past losers tend to outperform extreme past winners over long horizons (3–5 years)—termed *mean reversion*.

Recent work by Balvers and Wu (2006) and Koijen et al. (2009) demonstrates that momentum and mean reversion operate simultaneously at different timescales: short-term (1–3 months) momentum is followed by longer-term (6–12 month) mean reversion. This insight is critical for small-cap trading: the high volatility and lower institutional participation in small caps *amplify* both effects.

Moskowitz et al. (2012) documented momentum across asset classes (equities, commodities, currencies) and found it stronger in less liquid, smaller-capitalization markets—directly supporting the small-cap focus.

2.2 Technical Analysis and Pattern Recognition

While technical analysis (“charting”) has faced academic skepticism, recent work has legitimized certain patterns:

- **Breakout patterns** (Osler and Menkhoff, 2001): Breakouts through support/resistance are statistically significant predictors of short-term moves.
- **Range reversion** (Tversky and Kahneman, 1974; behavioral finance): Traders systematically overreact to news, creating mean-reversion opportunities.
- **Volume analysis** (Campbell and Hentschel, 1992): Unusual volume spikes correlate with volatility clustering and continuation patterns.

For small caps, technical analysis may be *more* reliable than fundamental analysis because information is priced more slowly; retail traders following simple, computable rules can capture this latency.

2.3 Microstructure and Retail Trading

Barclay and Warner (1993) and Harris (1994) established that market microstructure (bid-ask spread, order types, tick size) significantly impacts trading profitability. Small-cap stocks, with wider spreads and lower depth, require microstructure-aware strategies.

Recent work by Barclays Center for Electronic Markets and academic studies on retail trading (Boehmer et al., 2021) shows that retail traders, on aggregate, earn negative returns—but this is due to poor risk management and lack of discipline, not inherent unprofitability. Well-designed, algorithmic retail strategies can succeed.

2.4 Backtesting Pitfalls and Bias

Bailey et al. (2015) published an influential paper on “Pseudo-Mathematicians and Fake Data” demonstrating that ~95% of published trading strategies fail on out-of-sample data due to:

1. **Overfitting:** Parameters optimized on historical data do not generalize.
2. **Look-ahead bias:** Using information not available at decision time.
3. **Survivorship bias:** Ignoring delisted securities.
4. **Data snooping:** Running thousands of tests and reporting only winners (multiple testing problem).

This dissertation explicitly addresses each bias through:

- Nested cross-validation to control overfitting
- Walk-forward analysis with expanding training windows
- Point-in-time data handling (delisting aware)
- False Discovery Rate (FDR) control for multiple hypothesis testing
- Out-of-sample validation on unseen data

2.5 Risk Management in Retail Trading

Poundstone (2005) and other practitioner texts emphasize position sizing, stops, and exposure limits. Academic work by Campbell and Viceira (2002) formalizes dynamic risk management.

For small caps, traditional risk management must be adapted:

- **Hard stops** are unreliable (gaps, halts); instead, *time stops* and *structure-based stops* are preferred.
- **Kelly fraction** (optimal risk) is too aggressive for small caps; fractional Kelly (Kelly/4) is safer.
- **Volatility scaling** (risking a fixed dollar amount proportional to volatility) is critical.

2.6 Regulation and Compliance

The SEC’s Regulation SHO, short-sale locate requirements, and trading halts create operational constraints. The SEC’s 2019 guidance on electronic trading and market surveillance also imposes obligations on traders who deploy algorithmic systems.

2.7 Summary of Gaps Addressed by This Dissertation

Despite substantial literature on momentum, mean reversion, and backtesting rigor, gaps remain:

1. **Small-cap specificity:** Most academic work uses liquid large-cap data; small-cap dynamics differ significantly.
2. **Retail trading constraints:** Few papers address realistic transaction costs, liquidity filters, and borrow availability for retail traders.
3. **Reproducibility:** Most papers lack detailed code and data provenance; this dissertation emphasizes open-source, reproducible implementations.

4. **Ethical considerations:** Little discussion of manipulation risk and how strategies can be designed to avoid it.

This dissertation fills these gaps with rigorous, reproducible, ethically grounded work.

CHAPTER 3: DATA, UNIVERSE CONSTRUCTION, AND BIAS CONTROL

3.1 Data Sources and Tradeoffs

Multiple data sources are available for small-cap backtesting:

Data Source	Coverage	Cost	Adjusted Prices	Corporate Actions	Point-in-Time
Stooq (free)	US equities, delisted	Free	Yes	Yes	Limited
Polygon.io	US equities, real-time	\$99–\$199/mo	Yes	Yes	Yes
Norgate Data	US equities, delisted	\$99/mo	Yes (option)	Yes	Yes
CRSP	Academic license	\$5K–\$20K/yr	Yes	Yes	Yes
AlgoSeek	High-freq tick data	\$3K–\$10K/mo	No	Manual	Yes
Intrinio	US equities, fundamentals	\$99–\$349/mo	Yes	Yes	Yes

For this dissertation, we recommend:

- **Primary (free):** Stooq for initial development and backtests, accepting delisting bias as a transparency trade-off.
- **Production (paid):** Polygon.io or Norgate Data for live trading, due to explicit corporate action handling and point-in-time support.

3.2 Universe Definition

The small-cap universe is constructed as follows:

Definition (daily, as of market close):

$$U_t = \{\text{stock } s: \text{market_cap}_t \in [300M, 2B), \text{avg_volume}_{t-20} \geq 500K, \text{price}_t \in [1, 1000], \text{exchange} \in \{NYSE, NASDAQ\}, \text{ADR} = \text{False}\}$$

Rationale:

- **Market cap range:** Excludes penny stocks (<\$1) and illiquid microcaps; excludes mid-caps (>\$2B).
- **Average volume:** Ensures liquidity sufficient for retail order fills; \$500K minimum is conservative for small caps.
- **Price range:** Avoids penny stocks (manipulation risk) and inverse ETFs (structural decay).

- **Exchange:** US-listed only; includes ADRs exclusion to avoid FX complexity.

Universe refresh: Daily, based on most recent close.

3.3 Corporate Actions and Adjusted Prices

Splits, dividends, and other corporate actions corrupt price series. The standard approach:

1. **Stock splits:** Adjust historical prices backward: $P_{adj} = P_{nominal} / \text{split_factor}$.
2. **Dividends:** For this dissertation, use *price-only* returns (ignore dividends), as small caps pay dividends infrequently.
3. **Spinoffs, mergers, bankruptcies:** Mark as delisting on effective date.

Implementation in Python:

```
def apply_corporate_actions(df, splits_table, delisting_table):
```

```
    """
```

```
    Apply corporate actions to price series.
```

```
    Inputs:
```

```
    df: DataFrame with columns [date, symbol, open, high, low, close, volume]
```

```
    splits_table: DataFrame with [symbol, ex_date, split_factor]
```

```
    delisting_table: DataFrame with [symbol, delist_date]
```

```
    Returns:
```

```
    df: Adjusted price series
```

```
    """
```

```
    df = df.copy()
```

```
    # Apply splits backward
```

```
    for _, row in splits_table.iterrows():
```

```
        symbol, ex_date, factor = row['symbol'], row['ex_date'], row['split_factor']
```

```
        mask = (df['symbol'] == symbol) & (df['date'] < ex_date)
```

```
        for col in ['open', 'high', 'low', 'close']:
```

```
            df.loc[mask, col] /= factor
```

```
    # Mark delisted securities
```

```
    df['is_delisted'] = False
```

```
    for _, row in delisting_table.iterrows():
```

```
        symbol, delist_date = row['symbol'], row['delist_date']
```

```
        mask = (df['symbol'] == symbol) & (df['date'] >= delist_date)
```

```
        df.loc[mask, 'is_delisted'] = True
```

```
    return df
```

3.4 Bias-Control Mechanisms

3.4.1 Survivorship Bias

Definition: Ignoring delisted securities biases returns upward (survivors have outperformed delisted losers).

Control mechanism: Explicit delisting handling.

Implementation:

1. Maintain a “universe snapshot” at each historical date t : U_t includes all securities alive as of t (even if delisted later).
2. When computing portfolio returns on date $t + 1$, include all stocks in U_t , assigning zero return (or -100% if equity vaporizes) to delisted stocks.
3. This ensures no look-ahead bias.

Example (Python):

```
def compute_portfolio_returns_unbiased(universe_snapshots, prices, dates, delisting_dates):
```

```
    """
```

```
    Compute portfolio returns controlling for survivorship bias.
```

```
    Inputs:
```

```
    universe_snapshots: Dict mapping date -> set of tickers
```

```
    prices: DataFrame with [date, symbol, close]
```

```
    delisting_dates: Dict mapping symbol -> delist_date
```

```
    Returns:
```

```
    returns: Series of daily portfolio returns
```

```
    """
```

```
    returns = []
```

```
    for i in range(len(dates) - 1):
```

```
        date_t = dates[i]
```

```
        date_t1 = dates[i + 1]
```

```
        # Get universe as of date_t
```

```
        universe_t = universe_snapshots[date_t]
```

```
        daily_returns = []
```

```
        for symbol in universe_t:
```

```
            price_t = prices.loc[(prices['date'] == date_t) & (prices['symbol'] == symbol), 'close'].values
```

```
            price_t1 = prices.loc[(prices['date'] == date_t1) & (prices['symbol'] == symbol), 'close'].values
```

```
            if len(price_t) == 0 or len(price_t1) == 0:
```

```
                # Security delisted or missing data; assume 0 return
```

```
                daily_returns.append(0)
```

```
            elif symbol in delisting_dates and delisting_dates[symbol] <= date_t1:
```

```
                # Delisted on or before date_t1
```

```
                daily_returns.append(-1.0) # Total loss
```

```
            else:
```

```
                ret = (price_t1[0] - price_t[0]) / price_t[0]
```

```
                daily_returns.append(ret)
```

```
    portfolio_return = np.mean(daily_returns) # Equal-weighted for simplicity
```

```
    returns.append(portfolio_return)
```

```
return pd.Series(returns, index=dates[:-1])
```

3.4.2 Look-Ahead Bias

Definition: Using information not available at decision time (e.g., close prices for decisions made intraday).

Control mechanism: Strict temporal alignment.

Rules:

1. Signals computed on data available *as of the decision time*.
2. For backtests assuming daily closes: signals computed on $t - 1$ data; decisions made at t close.
3. For intraday backtests: signals computed on intraday data through the decision minute; fills assumed at the next minute's price (or next available).
4. Corporate action data: only use information public before the decision.

Example:

```
def generate_signals_no_lookahead(df_prices, decision_date, decision_hour=16):
```

```
    """
```

```
    Generate trading signals using only data available before decision_date/decision_hour.
```

```
    Inputs:
```

```
    df_prices: DataFrame with columns [timestamp, symbol, close]
```

```
    decision_date: datetime.date
```

```
    decision_hour: int (0-23)
```

```
    Returns:
```

```
    signals: Dict mapping symbol -> signal
```

```
    """
```

```
    cutoff = pd.Timestamp(decision_date) - pd.Timedelta(hours=decision_hour)
```

```
    df_prior = df_prices[df_prices['timestamp'] <= cutoff]
```

```
    signals = {}
```

```
    for symbol in df_prior['symbol'].unique():
```

```
        data = df_prior[df_prior['symbol'] == symbol].sort_values('timestamp')
```

```
        if len(data) >= 20: # Require 20 bars of history
```

```
            signal = compute_signal(data)
```

```
            signals[symbol] = signal
```

```
    return signals
```

3.4.3 Data-Snooping Bias (Overfitting)

Definition: Running thousands of parameter combinations and reporting only winners (multiple testing problem).

Control mechanism: Walk-forward analysis with nested cross-validation and False Discovery Rate (FDR) control.

Procedure:

1. **Outer loop (walk-forward):**
 - In-sample period: [2010-01-01, 2015-12-31] (6 years)
 - Out-of-sample test: 2016 (1 year)
 - Roll forward by 1 year; repeat until 2024.
2. **Inner loop (nested cross-validation):**
 - Within the in-sample period, divide into 5 folds.
 - For each fold: train on folds 1–4, validate on fold 5.
 - Optimize parameters on validation fold; refit on full in-sample; test on OOS year.
3. **Parameter selection:** Choose parameters that maximize mean validation Sharpe ratio (not best in-sample fit).
4. **FDR control:** If testing m independent strategy variants, apply Benjamini-Hochberg FDR control: report only strategies with p-value $< \alpha/m$.

Python implementation:

```
from sklearn.model_selection import TimeSeriesSplit

def walk_forward_validation(df_prices, param_grid, train_years=6, test_years=1):
    """
    Walk-forward validation with nested cross-validation.

    Inputs:
    df_prices: Time series of prices
    param_grid: Dict of parameter ranges
    train_years, test_years: Lengths of train/test windows

    Returns:
    results: List of (params, oos_sharpe, oos_returns)
    """
    dates = df_prices.index.unique()
    results = []

    for i in range(0, len(dates) - train_years - test_years, test_years):
        train_end = dates[i + train_years]
        test_end = dates[i + train_years + test_years]

        df_train = df_prices[:train_end]
        df_test = df_prices[train_end:test_end]

        # Nested cross-validation
        best_params = None
        best_val_sharpe = -np.inf
```

```

tscv = TimeSeriesSplit(n_splits=5)
for fold, (train_idx, val_idx) in enumerate(tscv.split(df_train)):
    df_fold_train = df_train.iloc[train_idx]
    df_fold_val = df_train.iloc[val_idx]

    for params in ParameterIterator(param_grid):
        strategy = Strategy(params)
        returns_val = strategy.backtest(df_fold_val)
        sharpe_val = np.mean(returns_val) / np.std(returns_val) * np.sqrt(252)

        if sharpe_val > best_val_sharpe:
            best_val_sharpe = sharpe_val
            best_params = params

# Refit on full training set
strategy = Strategy(best_params)
returns_oos = strategy.backtest(df_test)
sharpe_oos = np.mean(returns_oos) / np.std(returns_oos) * np.sqrt(252)

results.append({
    'train_period': (dates[i], train_end),
    'test_period': (train_end, test_end),
    'params': best_params,
    'oos_sharpe': sharpe_oos,
    'oos_returns': returns_oos
})

return results

```

3.4.4 Leakage from Adjusted Prices

Definition: Using post-split/post-dividend prices to make decisions, then filling at pre-adjusted prices (or vice versa).

Control mechanism: Consistent use of adjusted prices.

Rule: All price series (for signal generation, filling, analysis) use consistently adjusted prices. If using “natural” (unadjusted) prices for execution, adjust the decision thresholds accordingly.

3.5 As-Of Data Processing

“As-of” joins are critical for point-in-time data handling.

Scenario: We have a prices table (daily closes) and a fundamentals table (quarterly earnings, book value). When making a decision on date t , we should use the most recent fundamentals reported before t (avoiding look-ahead).

SQL example:

```
SELECT
    p.date, p.symbol, p.close, f.earnings, f.book_value
FROM prices p
LEFT JOIN LATERAL (
    SELECT earnings, book_value
    FROM fundamentals f
    WHERE f.symbol = p.symbol
    AND f.report_date < p.date
    ORDER BY f.report_date DESC
    LIMIT 1
) f ON true
WHERE p.date >= '2015-01-01' AND p.date <= '2024-12-31'
ORDER BY p.date, p.symbol
```

CHAPTER 4: METHODOLOGY & STATISTICAL VALIDATION

4.1 Walk-Forward Analysis

Walk-forward analysis simulates realistic market conditions by rolling a training window forward through time, re-optimizing parameters periodically, and testing on unseen data.

Formal definition:

For time series $\mathcal{D} = \{(t_1, x_1), \dots, (t_N, x_N)\}$:

1. Choose W (window size, e.g., 6 years) and S (step size, e.g., 1 year).
2. For $i = 0, 1, \dots, \lfloor (N - W)/S \rfloor$:
 - **Training set:** $\mathcal{D}_i^{\text{train}} = \{(t_j, x_j) : j \in [iS, iS + W]\}$
 - **Test set:** $\mathcal{D}_i^{\text{test}} = \{(t_j, x_j) : j \in [iS + W, iS + W + S]\}$
 - **Procedure:**
 - a. Optimize strategy parameters on $\mathcal{D}_i^{\text{train}}$ (using nested CV)
 - b. Apply optimized parameters to $\mathcal{D}_i^{\text{test}}$ (no re-optimization)
 - c. Record out-of-sample Sharpe ratio, max drawdown, and other metrics
3. **Final report:** Aggregate metrics across all OOS test periods.

Advantages:

- Simulates parameter drift (re-optimization every period)
- Avoids overfitting (parameters frozen on test period)
- Realistic transaction costs (each re-opt incurs slippage)

4.2 Nested Cross-Validation

Within each training window, use nested cross-validation to select parameters without contaminating the test set.

Procedure:

1. **Outer loop:** Walk-forward windows (above).
2. **Inner loop (within training set):**
 - Divide $\mathcal{D}_i^{\text{train}}$ into K folds (e.g., $K = 5$).
 - For each fold k :
 - Train on folds $\{1, \dots, K\} \setminus \{k\}$
 - Validate on fold k
 - Record validation metric (e.g., Sharpe ratio)
3. **Parameter selection:** Choose parameters maximizing mean validation metric across folds.
4. **Refit:** Refit chosen parameters on entire $\mathcal{D}_i^{\text{train}}$ and test on $\mathcal{D}_i^{\text{test}}$.

4.3 Risk Metrics and Definitions

4.3.1 Return Metrics

Cumulative return:

$$R_{\text{cum}} = \prod_{t=1}^T (1 + r_t) - 1$$

where r_t is the return on day t .

Annualized return:

$$R_{\text{annual}} = (1 + R_{\text{cum}})^{252/T} - 1$$

where T is the number of trading days.

Monthly/annual returns distribution: Report percentiles ($p_{10}, p_{25}, \text{median}, p_{75}, p_{90}$) and skewness, kurtosis.

4.3.2 Volatility and Sharpe Ratio

Daily volatility:

$$\sigma_{\text{daily}} = \sqrt{\frac{1}{T-1} \sum_{t=1}^T (r_t - \bar{r})^2}$$

Annualized volatility:

$$\sigma_{\text{annual}} = \sigma_{\text{daily}} \cdot \sqrt{252}$$

Sharpe ratio (excess return per unit risk):

$$\text{Sharpe} = \frac{R_{\text{annual}} - R_f}{\sigma_{\text{annual}}}$$

where R_f is the risk-free rate (assume 3% on 10-year Treasury).

4.3.3 Maximum Drawdown

Definition: Largest cumulative loss from a peak.

$$\text{DD}_{\text{max}} = \min_t \left(\frac{L_t - \max_{s \leq t} L_s}{\max_{s \leq t} L_s} \right)$$

where L_t is the cumulative portfolio value on day t .

Interpretation: If an account reaches a \$1M peak, then drops to \$900K, the drawdown is -10%.

Calmar ratio: $\text{Calmar} = R_{\text{annual}} / |\text{DD}_{\text{max}}|$ (return per unit drawdown risk).

4.3.4 Value at Risk (VaR) and Conditional VaR (CVaR)

Historical VaR at confidence level α :

$$\text{VaR}_\alpha = \text{quantile}(\{r_1, \dots, r_T\}, 1 - \alpha)$$

For example, $\text{VaR}_{0.95}$ is the 5th percentile of returns (daily loss exceeded with 5% probability).

Conditional VaR (Expected Shortfall):

$$\text{CVaR}_\alpha = \mathbb{E}[r_t | r_t \leq \text{VaR}_\alpha]$$

Average loss on worst $\alpha \times 100\%$ of days.

Reporting: Report VaR and CVaR at 95% and 99% levels.

4.3.5 Trade-Level Metrics

Win rate:

$$W = \frac{\text{\#profitable trades}}{\text{\#total trades}}$$

Profit factor:

$$\text{PF} = \frac{\sum \text{profits (winning trades)}}{\sum |\text{losses (losing trades)}|}$$

Average trade duration, average win size, average loss size: Descriptive statistics on trade-level returns.

Expectancy (average profit per trade):

$$E = W \times \text{avg_win} + (1 - W) \times \text{avg_loss}$$

4.4 Hypothesis Testing and Multiple Testing Correction

4.4.1 T-Tests for Strategy Performance

Test null hypothesis H_0 : strategy returns are zero (excess return statistically insignificant).

t-statistic:

$$t = \frac{\bar{r} - 0}{\text{SE}(\bar{r})} = \frac{\bar{r}}{\sigma/\sqrt{T}}$$

where $\bar{r}$ is mean daily return, σ is daily volatility, and T is the number of days.

Critical issues:

1. **Non-normality:** Small-cap returns are non-normal (fat tails, skew). Use bootstrap t-tests instead of Student's t.
2. **Autocorrelation:** Daily returns exhibit autocorrelation; use Newey-West standard errors.

Bootstrap t-test (implementation):

```
def bootstrap_t_test(returns, n_bootstrap=10000):  
    """  
    Compute t-statistic and p-value using bootstrap (non-parametric).  
  
    Inputs:  
    returns: Array of daily returns  
    n_bootstrap: Number of bootstrap resamples  
  
    Returns:  
    t_stat, p_value  
    """  
    t_obs = np.mean(returns) / (np.std(returns, ddof=1) / np.sqrt(len(returns)))  
  
    t_boots = []  
    for _ in range(n_bootstrap):  
        resampled = np.random.choice(returns, size=len(returns), replace=True)  
        t_boot = np.mean(resampled) / (np.std(resampled, ddof=1) / np.sqrt(len(resampled)))  
        t_boots.append(t_boot)  
  
    p_value = np.mean(np.abs(np.array(t_boots)) > np.abs(t_obs))  
    return t_obs, p_value
```

4.4.2 Multiple Hypothesis Testing Correction

If testing m independent strategies, probability of false positive is inflated.

Bonferroni correction: Reject H_0 if $p < \alpha/m$.

Benjamini-Hochberg FDR control: 1. Sort p-values: $p_{(1)} \leq \dots \leq p_{(m)}$. 2. Find largest i such that $p_{(i)} \leq (i/m)\alpha$. 3. Reject H_0 for strategies $1, \dots, i$.

FDR is less conservative than Bonferroni, controlling the false discovery rate (not the family-wise error rate).

Implementation:

```
def fdr_control(p_values, alpha=0.05):  
    """  
    Apply Benjamini-Hochberg FDR control.  
  
    Inputs:  
    p_values: Array of p-values for m tests  
    alpha: Desired FDR level  
  
    Returns:  
    rejected: Array of boolean, True if null rejected  
    """  
    m = len(p_values)  
    sorted_idx = np.argsort(p_values)
```

```

sorted_p = p_values[sorted_idx]

rejected = np.zeros(m, dtype=bool)

for i in range(m - 1, -1, -1):
    if sorted_p[i] <= (i + 1) / m * alpha:
        rejected[sorted_idx[i+1]] = True
        break

return rejected

```

4.5 Why Many Backtests Fail: A Diagnostic Checklist

Common failures in retail trading backtests:

Issue	Impact	Detection	Remedy
Survivorship bias	+2% to +10% return overstatement	OOS testing on delisted securities	Use point-in-time universe snapshots
Look-ahead bias	+5% to +50% overstatement (severe)	Compare decision time with data cutoff	Strict temporal alignment; 1-minute lag
Overfitting	OOS Sharpe degrades by 50–90%	Compare in-sample vs. OOS metrics	Walk-forward with nested CV
Spread slippage	−0.5% to −2% annual return	Compare backtest to live trading	Model realistic spreads (200+ basis points)
Liquidity assumption	Fill at limit price never happens	Check volume at price levels	Require min 2x daily volume; conservative fills
Gap fill	Assume fills after gap (impossible)	Chart prices around gap dates	Assume gap fills at opening, not limit
Survivor winners only	~95% of published results fail	Walk-forward OOS; FDR control	Nested CV; Bonferroni / FDR
Hidden market impact	Large orders move price against you	Assume fills degrade with size	Use 10% of daily volume max
Regime change	Strategy breaks in new market	Backtest across 2008, 2020, 2022 crises	Stress test; regime-conditional metrics

CHAPTER 5: STRATEGY FAMILIES—DETAILED SPECIFICATIONS

This chapter provides precise mathematical definitions, entry/exit rules, feature engineering, risk controls, and pseudocode for six complementary strategy families.

5.1 Strategy Family A: Volatility-Scaled Small-Cap Momentum with Liquidity Filters

5.1.1 Concept

Capitalize on short-term (1–4 week) momentum effect in small caps while controlling exposure for volatility. Small-cap volatility is mean-reverting; the strategy scales position size inversely with volatility to maintain consistent risk.

5.1.2 Signal Definition

Momentum signal: Average returns over a lookback window.

$$\text{MOM}_t = \frac{\sum_{i=1}^L r_{t-i}}{L}$$

where L is the lookback window (e.g., $L = 20$ days).

Volatility (annualized):

$$\sigma_t = \sqrt{\text{mean}(r_{t-20}^2) \cdot \sqrt{252}}$$

Liquidity score:

$$\text{LIQ}_t = \text{volume}_t \times \text{close}_t$$

Entry condition: - $\text{MOM}_t > \tau_{\text{mom}}$ (positive momentum, e.g., $\tau_{\text{mom}} = 0.5\%$) - $\text{LIQ}_t > \$500K$ (daily volume $\times$ price) - $\sigma_t < 1.5$ (volatility below threshold; high volatility stocks excluded) - Price in $[\$2, \$50]$ (avoid penny stocks, expensive growth)

Exit conditions: 1. **Profit target:** Price reaches entry + ATR_t (where $\text{ATR} = \text{Average True Range}$) 2. **Stop loss:** Time-based: 10 trading days from entry, or price drops $2 \times \text{ATR}$ 3. **Risk control:** Exit if intraday loss exceeds 2% of position

5.1.3 Position Sizing: Volatility-Scaled

$$\text{position_size}_t = \frac{R}{\sigma_t \times \text{price}_t}$$

where R is the target risk per trade (e.g., \$200), and σ_t is annualized volatility.

Interpretation: Higher volatility $\rightarrow$ smaller position size (in dollars) to keep dollar risk constant.

Example: If a stock has \$50 price and 80% annualized volatility:

$$\text{position_size} = \frac{\$200}{0.80 \times \$50} = 5 \text{ shares}$$

5.1.4 Pseudocode

algorithm VoL_SCALED_MOMENTUM

input: daily prices, volume, parameter L (lookback)

output: position, entry_price, exit_rules

for each trading day t:

 for each stock s in universe:

 // Compute momentum

 momentum = mean(returns[-L:])

 // Compute volatility

 volatility = std(returns[-20:]) * sqrt(252)

 // Check liquidity

 avg_volume = mean(volume[-20:])

 dollar_volume = avg_volume * close[t]

 if momentum > 0.005 AND dollar_volume > 500000 AND volatility < 1.5 AND price in [2, 50]:

 // Signal: long

 position_size = 200 / (volatility * close[t])

 entry_price = close[t] + spread_adjustment

 // Set risk levels

 atr = compute_atr(high, low, 14)

 profit_target = entry_price + atr

 stop_loss = entry_price - 2*atr

 time_stop = today + 10 days

 place_order(LONG, position_size, entry_price)

 // Check exits for open positions

 if position.open AND position.entry_date <= today - 10:

 close_position(reason="time_stop")

 else if price < position.stop_loss:

 close_position(reason="stop_loss")

 else if price > position.profit_target:

 close_position(reason="profit_target")

end for

5.1.5 Feature Engineering Details

ATR (Average True Range):

$$TR_t = \max(\text{high}_t - \text{low}_t, |\text{high}_t - \text{close}_{t-1}|, |\text{low}_t - \text{close}_{t-1}|)$$

$$ATR_t = \text{SMA}(TR, 14)$$

Rolling standard deviation (for volatility):

```
def compute_rolling_volatility(closes, window=20):
    returns = np.diff(np.log(closes)) * 100 # Log returns in percentage
    rolling_std = pd.Series(returns).rolling(window).std()
    annualized_vol = rolling_std.values * np.sqrt(252)
    return annualized_vol
```

5.1.6 Parameter Ranges and Overfitting Control

Parameter	Range	Recommended	Rationale
Lookback (L)	10–40 days	20	Captures short-term momentum; longer = noisier
Momentum threshold	0.2% – 1.0%	0.5%	Too high = few signals; too low = noise
Volatility cap	1.0 – 2.5	1.5	Higher vol → lower signal quality
Stop loss (ATR multiple)	1.5 – 3.0	2.0	Tighter stops reduce capital at risk; looser = larger losses
Time stop (days)	5 – 20	10	Longer = tail risk; shorter = miss continuation

Optimization: Use nested CV within walk-forward to select parameters. Do not exhaustively search; instead, use 2–3 parameter combinations and report results for each.

5.2 Strategy Family B: Gap-and-Go / Opening Range Breakout (ORB) with Robust Filters

5.2.1 Concept

Small-cap stocks commonly gap up or down at open, especially after significant overnight news. This strategy exploits opening range breakouts while filtering false breakouts using volume and volatility.

5.2.2 Signal Definition

Gap:

$$\text{gap} = \frac{\text{open}_t - \text{close}_{t-1}}{\text{close}_{t-1}} \times 100$$

Opening range (first 30 minutes):

$$\text{ORH} = \max(\text{high}[9:30, 10:00])$$

$$\text{ORL} = \min(\text{low}[9:30, 10:00])$$

Opening range width:

$$\text{ORB_range} = \text{ORH} - \text{ORL}$$

Volume confirmation: - Require opening 30-min volume to exceed 50% of average daily volume. - Prevent low-volume false breaks.

Entry conditions (for long): 1. Gap up: $\text{gap} > 2\%$ 2. Price breaks above ORH by \$0.05 (avoiding exact top) 3. Volume in breakout bar $\geq 0.5 \times \text{avg_daily_volume}$ 4. Bid-ask spread $< \$0.05$ (liquidity available)

Exit conditions: 1. **Profit target:** $\text{Entry} + 2 \times \text{ORB_range}$ (or ATR) 2. **Time stop:** 15:30 (end of trading; avoid overnight gap risk) 3. **Stop loss:** Below ORB_range if price reverses below opening range

5.2.3 Pseudocode

algorithm ORB_STRATEGY

input: minute-level OHLCV for first 30 min and full day

output: signals, entry_price, exit_rules

for each trading day t:

 // Compute gap

$\text{gap} = (\text{open}[9:30] - \text{close}[t-1]) / \text{close}[t-1]$

 if $\text{gap} > 0.02$: // Gap up

 // Compute opening range (first 30 min)

$\text{minutes_0930_1000} = \text{filter}(\text{timestamps in } [9:30, 10:00])$

$\text{orh} = \max(\text{high}[\text{minutes_0930_1000}])$

$\text{orl} = \min(\text{low}[\text{minutes_0930_1000}])$

$\text{orb_range} = \text{orh} - \text{orl}$

 // Wait for breakout confirmation

 for t_bar in range(10:01, 15:30, 1 minute):

 if $\text{high}[t_bar] > \text{orh} + 0.05$:

 // Potential breakout

$\text{volume_bar} = \text{volume}[t_bar]$

$\text{avg_volume_daily} = \text{mean}(\text{volume}[-20:])$

 if $\text{volume_bar} \geq 0.5 * \text{avg_volume_daily}$ AND $\text{bid_ask_spread} < 0.05$:

 // Entry confirmed

$\text{entry_price} = \text{orh} + 0.05$

$\text{target} = \text{entry_price} + 2 * \text{orb_range}$

$\text{stop} = \text{orl} - 0.05$

$\text{place_order}(\text{LONG}, \text{size}, \text{entry_price})$

 // Monitor exits

 while $\text{time} < 15:30$:

 if $\text{price} > \text{target}$:

$\text{close_position}(\text{reason} = \text{"profit_target"})$

```

        break
    else if price < stop:
        close_position(reason="stop_loss")
        break

    if time >= 15:30:
        close_position(reason="end_of_day")

end for

```

5.2.4 Risk Management: Avoiding False Breaks

False break detection: If price breaks ORH but reverses below ORB_range within 5 minutes, exit immediately.

```

def detect_false_breakout(prices_minute, orh, orl, lookback_minutes=5):
    """
    Detect false breakouts (break above ORH but reverse into ORB).
    """
    if len(prices_minute) < lookback_minutes:
        return False

    recent_low = min(prices_minute[-lookback_minutes:])
    return recent_low < orl

```

5.3 Strategy Family C: Mean Reversion After Extreme Intraday Moves

5.3.1 Concept

Small-cap stocks exhibit intraday overshoots due to retail traders' emotional reactions and low liquidity. This strategy buys extreme intraday dips and sells extreme intraday highs, betting on reversion within the day.

5.3.2 Signal Definition

Daily open (reference):

$$O_t = \text{open price at 9:30}$$

Intraday extreme move (% from open):

$$\text{intraday_move} = \frac{\text{price}_t - O_t}{O_t} \times 100$$

Intraday volatility (realized):

$$IV_t = (\text{high}_t - \text{low}_t) / O_t$$

Bollinger Bands (on intraday prices):

$$\text{BB_mid} = \text{SMA}(\text{price}, 20)$$

$$\text{BB_upper} = \text{BB_mid} + 2 \times \text{std}(\text{price}, 20)$$

$$BB_lower = BB_mid - 2 \times std(price, 20)$$

Entry conditions (for long reversion buy): 1. Intraday move: $intraday_move < -3\%$ (down 3%+ from open) 2. Price touches or crosses BB_lower (extreme) 3. Time: 10:00 – 15:00 (sufficient time to revert; not end of day) 4. Volume spike: Last 5-min volume $> 1.5 \times$ avg 5-min volume

Exit conditions: 1. **Profit target:** Entry + 1.5% to 2% 2. **Stop loss:** Entry - 1% (intraday whipsaw protection) 3. **Time stop:** 15:45 (close of day)

5.3.3 Pseudocode

algorithm INTRADAY_MEAN_REVERSION

input: minute-level OHLCV

output: signals, positions

daily_open = open[9:30]

bb_lookback = 20 minutes

for t_minute in [10:00, 15:45]:

 current_price = close[t_minute]

 intraday_move_pct = (current_price - daily_open) / daily_open * 100

 // Compute Bollinger Bands on last 20 minutes

 prices_recent = close[t_minute - 20:t_minute]

 bb_mid = mean(prices_recent)

 bb_std = std(prices_recent)

 bb_lower = bb_mid - 2 * bb_std

 bb_upper = bb_mid + 2 * bb_std

 // Check for extreme dip

 if intraday_move_pct < -3.0 AND current_price <= bb_lower:

 volume_recent = mean(volume[t_minute - 5:t_minute])

 if volume_recent > 1.5 * avg_5min_volume:

 // Entry signal: mean reversion buy

 entry_price = current_price

 target = entry_price * 1.015 // +1.5%

 stop = entry_price * 0.99 // -1%

 place_order(LONG, size, entry_price)

 // Monitor until close

 while t <= 15:45:

 if price >= target:

 close_position(reason="profit_target")

 break

 else if price < stop:

 close_position(reason="stop_loss")

 break

if t >= 15:45:

```
close_position(reason="end_of_day")
```

end for

5.3.4 Volatility Clustering and Risk Control

Mean reversion works best when volatility is elevated (more likely to revert). If IV is very low, reversion may not occur within the day.

IV-contingent entry:

```
def filter_by_iv(price_series_intraday, min_iv_pct=1.5, max_iv_pct=5.0):  
    """  
    Filter entries by intraday volatility range.  
    Too low: no momentum; too high: too risky.  
    """  
    intraday_vol = (max(price_series_intraday) - min(price_series_intraday)) / price_series_intraday[0]  
    return min_iv_pct / 100 <= intraday_vol <= max_iv_pct / 100
```

5.4 Strategy Family D: Breakout and Retest Pattern Recognition

5.4.1 Concept

A classic technical pattern: price breaks above resistance, pulls back (retest), and continues upward. This strategy algorithmically detects and trades the retest.

5.4.2 Resistance/Support Levels

Method 1: Pivot levels (from prior day):

$$\text{pivot} = \frac{\text{high}_{t-1} + \text{low}_{t-1} + \text{close}_{t-1}}{3}$$

$$\text{resistance} = 2 \times \text{pivot} - \text{low}_{t-1}$$

$$\text{support} = 2 \times \text{pivot} - \text{high}_{t-1}$$

Method 2: N-day highs/lows:

$$\text{resistance} = \max(\text{high}[t - 20:t])$$

$$\text{support} = \min(\text{low}[t - 20:t])$$

5.4.3 Breakout and Retest Logic

Phase 1: Detect breakout - Price closes above resistance with volume confirmation (e.g., volume > 120% of average). - Wait for next day.

Phase 2: Detect retest - Price pulls back to 50–80% of the prior day's range (retest of resistance, now becoming support). - Volume on retest is lower (profit-taking pullback, not reversal).

Phase 3: Continue above - Price breaks above retest level with renewed volume. - Entry signal.

5.4.4 Pseudocode

algorithm BREAKOUT_RETEST

input: daily OHLCV, resistance_method

output: signals

state_tracker = {} // Track breakouts per stock

for each trading day t:

 for stock s:

 // Compute resistance/support

 if resistance_method == "PIVOT":

 pivot = (high[t-1] + low[t-1] + close[t-1]) / 3

 resistance = 2*pivot - low[t-1]

 support = 2*pivot - high[t-1]

 else: // N-day high

 resistance = max(high[t-20:t-1])

 support = min(low[t-20:t-1])

 // Check for breakout

 if close[t] > resistance AND volume[t] > 1.2 * avg_volume[t-20:t]:

 state_tracker[s] = {"phase": "BREAKOUT", "breakout_date": t, "resistance": resistance}

 // Check for retest (on day after breakout)

 elif s in state_tracker AND state_tracker[s]["phase"] == "BREAKOUT":

 if state_tracker[s]["breakout_date"] == t - 1:

 range_prev = high[t-1] - low[t-1]

 retest_level = resistance - 0.5 * range_prev

 if low[t] <= retest_level AND close[t] > support:

 state_tracker[s]["phase"] = "RETEST"

 state_tracker[s]["retest_level"] = retest_level

 // Check for continuation (above retest)

 elif s in state_tracker AND state_tracker[s]["phase"] == "RETEST":

 if close[t] > state_tracker[s]["retest_level"] AND volume[t] > avg_volume:

 // Entry signal

 entry_price = state_tracker[s]["retest_level"]

 target = resistance + (resistance - support) // Measured move

 stop = support - 0.05

 place_order(LONG, size, entry_price)

 state_tracker[s]["phase"] = "ENTERED"

end for

5.4.5 Risk Management

Stop loss: Placed below the support level (or retest level).

Profit target: Measured move (distance of breakout above resistance = distance expected for continuation).

5.5 Strategy Family E: Event-Driven Volatility Strategy (Earnings Announcement)

5.5.1 Concept

Earnings announcements create volatility spikes. This strategy buys strangle (long call + long put) expiring post-earnings, profiting from volatility expansion regardless of direction.

Note: For simplicity, this section focuses on equity-based event trading (betting on price range expansion) rather than options.

5.5.2 Equity-Based Event Strategy (Earnings Surprise)

Pre-earnings signal: - Identify stocks with earnings announcement on date E . - Compute “surprise factor” from options implied volatility (if available) or historical vol. - **Expected move:** IV implied move or $2 \times (20\text{-day ATR})$.

Entry conditions (day before earnings): 1. Place limit buy orders at (close - $0.6 \times \text{expected move}$) and above (close + $0.6 \times \text{expected move}$). 2. Expect fills if earnings surprise moves stock sharply.

Timestamp discipline (critical): - Earnings announcement times vary: after hours (4 PM ET), after next open, etc. - Use point-in-time earnings calendar data (e.g., from EDGAR). - Do not use forward-looking data.

5.5.3 Python Implementation Example

```
def event_driven_earnings_entry(stock_symbol, earnings_date, historical_returns, iv_implied_move=None):
```

```
    """
```

```
    Generate entry orders for earnings event.
```

```
    Inputs:
```

```
    stock_symbol: str
```

```
    earnings_date: date of announcement
```

```
    historical_returns: past 20 days of returns
```

```
    iv_implied_move: float, implied move from options (if available)
```

```
    Returns:
```

```
    entry_orders: dict with buy and sell limits
```

```
    """
```

```
    current_price = historical_returns[-1]
```

```
    # Compute expected move
```

```
    if iv_implied_move:
```

```
        expected_move = iv_implied_move
```

```
    else:
```

```
        historical_vol = np.std(historical_returns) * np.sqrt(252)
```

```
        expected_move = current_price * historical_vol / 20 # Rough approximation
```

```

# Entry: 60% of expected move
buy_limit = current_price - 0.6 * expected_move
sell_limit = current_price + 0.6 * expected_move

# Position sizing: equal risk on both sides
position_size = int(500 / expected_move) # $500 risk per side

entry_orders = {
    'buy': {'price': buy_limit, 'size': position_size, 'type': 'limit'},
    'sell': {'price': sell_limit, 'size': position_size, 'type': 'limit'},
    'expected_move': expected_move,
    'earnings_date': earnings_date
}

return entry_orders

```

5.5.4 Exit Strategy Post-Earnings

- If earnings surprise (price moves $> 1 \times$ expected move), hold for 1–2 days (post-event momentum).
- If no surprise, exit within 24 hours of announcement.
- Use time stops to avoid overnight risk.

5.6 Strategy Family F: Market Regime-Filtered Composite Strategy

5.6.1 Concept

Small-cap trading success is regime-dependent: - **Risk-on regime:** Momentum strategies work well. - **Risk-off regime:** Mean reversion or low-volatility strategies work; shorts are dangerous.

This strategy dynamically selects from the above families based on market regime.

5.6.2 Regime Definition

Risk-on indicator:

$$\text{regime} = \text{sign}(\text{SPY}_t - \text{SMA}(\text{SPY}, 50))$$

If SPY is above its 50-day moving average $\rightarrow$ risk-on (momentum strategies active). If SPY is below its 50-day moving average $\rightarrow$ risk-off (mean reversion strategies active).

VIX-based regime (alternative):

$$\text{regime} = \begin{cases} \text{risk-on} & \text{if } \text{VIX} < 20 \\ \text{risk-off} & \text{if } \text{VIX} \geq 20 \end{cases}$$

5.6.3 Composite Strategy Selection Logic

algorithm REGIME_FILTERED_COMPOSITE

input: daily SPY prices, small-cap universe prices, VIX

for each trading day t:

 // Determine regime

 sma_50_spy = SMA(SPY, 50)

```

if SPY[t] > sma_50_spy:
    regime = "RISK_ON"
else:
    regime = "RISK_OFF"

if regime == "RISK_ON":
    // Activate momentum and breakout strategies
    signals_momentum = VOLATILITY_SCALED_MOMENTUM.generate_signals(...)
    signals_orb = ORB_STRATEGY.generate_signals(...)
    signals = combine(signals_momentum, signals_orb)

else: // RISK_OFF
    // Activate mean reversion strategy
    signals = INTRADAY_MEAN_REVERSION.generate_signals(...)

// Execute signals
for stock, signal in signals:
    if signal["type"] == "LONG" AND portfolio.cash > signal["required_capital"]:
        place_order(LONG, signal["size"], signal["entry_price"])

end for

```

5.6.4 Rationale

By filtering strategies by regime, we reduce false signals in unfavorable market conditions. Momentum breaks down in downtrends; mean reversion fails in strong uptrends.

CHAPTER 6: RISK MANAGEMENT FRAMEWORK

6.1 Position Sizing Methodologies

6.1.1 Fixed Fractional Sizing

Risk a fixed fraction of account equity on each trade.

$$\text{position_value} = \text{account_equity} \times f$$

where f is the fractional risk (e.g., 2% $\rightarrow f = 0.02$).

Adjustment for stop loss:

$$\text{position_size} = \frac{\text{position_value}}{\text{stop_loss_distance}}$$

Example: - Account: \$100K - Risk fraction: 2% $\rightarrow$ risk \$2K per trade - Stop loss: 2% from entry - Position size: \$2000 / 0.02 = \$100K notional (full account)

Issue: Over-leverages on tight stops; under-leverages on wide stops.

6.1.2 Volatility-Parity Sizing

Adjust position size so expected loss is constant across all trades, regardless of asset volatility.

$$\text{position_size} = \frac{R}{\sigma_t \times \text{price}_t}$$

where: - R = target risk per trade (e.g., \$200) - σ_t = annualized volatility - price_t = current price

Example: - Stock A: price = \$50, vol = 80% $\rightarrow$ position = \$200 / (0.80 $\times$ 50) = 5 shares - Stock B: price = \$20, vol = 40% $\rightarrow$ position = \$200 / (0.40 $\times$ 20) = 25 shares

Advantage: Homogeneous risk across trades; prevents concentrated bets on volatile names.

6.1.3 Kelly Criterion (with caution)

Optimal fraction of bankroll to risk on a favorable bet.

$$f^* = \frac{p \times b - q}{b}$$

where: - p = win probability - $q = 1 - p$ = loss probability - b = win/loss ratio (avg win size / avg loss size)

Example: - Win rate: 55% - Win/loss ratio: 1.5 - $f^* = (0.55 \times 1.5 - 0.45)/1.5 = (0.825 - 0.45)/1.5 = 0.25 = 25\%$

Caution: Kelly can be too aggressive for trading (leads to severe drawdowns even if profitable). Use fractional Kelly (Kelly/2 or Kelly/4) instead.

def kelly_fraction(win_rate, avg_win, avg_loss, kelly_divisor=4):

"""

Compute Kelly fraction with divisor for safety.

Inputs:

win_rate: float, proportion of winning trades
avg_win: float, average \$ profit
avg_loss: float, average \$ loss
kelly_divisor: int, conservative divisor (2, 4, or higher)

Returns:

optimal_fraction: float, percentage of capital to risk
"""

```
if avg_loss <= 0:  
    return 0
```

```
b = avg_win / avg_loss # Win/loss ratio  
p = win_rate  
q = 1 - win_rate
```

```
kelly_optimal = (p * b - q) / b  
kelly_safe = max(0, kelly_optimal / kelly_divisor) # Fractional Kelly
```

```
return kelly_safe
```

6.1.4 Drawdown-Based Adaptive Sizing

Reduce position size if cumulative drawdown exceeds threshold (risk proportional to prior losses).

$$\text{position_multiplier} = \begin{cases} 1.0 & \text{if } DD < 10\% \\ 0.75 & \text{if } 10\% \leq DD < 15\% \\ 0.5 & \text{if } 15\% \leq DD < 20\% \\ 0.25 & \text{if } DD \geq 20\% \end{cases}$$

```
def adaptive_position_multiplier(drawdown_pct):  
    """
```

```
    Reduce position size based on current drawdown.  
    """
```

```
    if drawdown_pct < 0.10:  
        return 1.0  
    elif drawdown_pct < 0.15:  
        return 0.75  
    elif drawdown_pct < 0.20:  
        return 0.50  
    else:  
        return 0.25
```

6.2 Portfolio Constraints

6.2.1 Gross Exposure Cap

Limit total notional exposure to avoid overleveraging.

$$\sum_i |\text{position_size}_i \times \text{price}_i| \leq 2 \times \text{account_equity}$$

(2× is a common limit for retail margin; 1× for fully paid accounts.)

6.2.2 Sector Caps

Prevent concentration in a single sector (small-cap biotech, fintech, energy, etc.).

$$\text{sector_exposure} \leq 20\% \times \text{gross_exposure}$$

Example: If gross exposure is \$200K, no single sector (e.g., biotech) exceeds \$40K notional.

6.2.3 Correlation Control (portfolio hedging)

Compute rolling correlation between open positions. If correlation exceeds threshold, reduce size of correlated trades.

```
def portfolio_correlation_matrix(open_positions, price_history, lookback=30):
    """
    Compute correlation matrix of open positions.
    """
    symbols = [p['symbol'] for p in open_positions]
    prices = price_history[symbols].iloc[-lookback:].pct_change()
    corr_matrix = prices.corr()
    return corr_matrix
```

6.3 Stop-Loss Policies: Small-Cap Specific Considerations

6.3.1 Hard Stops vs. Time Stops

Hard stops (price-based): - Exit if price drops $X\%$ from entry. - **Problem in small caps:** Gaps, halts, and illiquidity mean stops don't execute at the specified level. A 2% stop might execute at -5%.

Time stops (calendar-based): - Exit after N days regardless of price. - **Advantage:** Guaranteed execution; prevents zombie positions. - **Disadvantage:** May exit winners prematurely.

Hybrid approach (recommended for small caps): - Use hard stops for daily moves (2–3%). - Use time stops (5–10 days) to limit tail risk. - Use volatility-based stops (ATR multiples).

6.3.2 ATR-Based Stops

$$\text{stop_price} = \text{entry} - K \times \text{ATR}_{14}$$

where $K \in [1.5, 2.5]$.

Advantage: Adapts to market volatility; wider stops in volatile markets, tighter in quiet markets.

6.3.3 Structure-Based Stops

Identify key support/resistance levels from chart patterns; place stops below support.

```
def structure_based_stop(support_level, entry_price, margin=0.02):
    """
    Place stop below support level.
    """
    stop = support_level - margin # Small margin below support
    return stop
```

6.4 Disaster Controls

6.4.1 Daily Loss Limit

If cumulative P&L loss exceeds threshold (e.g., −2% of account), halt all new trades for the remainder of the day.

```
def check_daily_loss_limit(daily_pnl, account_equity, limit_pct=0.02):
    """
    Check if daily loss exceeds limit.
    """
    loss_limit = account_equity * limit_pct
    if daily_pnl < -loss_limit:
        return "TRADING_HALTED"
    return "TRADING_ALLOWED"
```

6.4.2 Circuit Breaker

Stop all trading if individual position loss exceeds threshold (e.g., −5% on a single position).

```
def circuit_breaker(position_pnl, position_value, limit_pct=0.05):
    """
    Emergency kill-switch for individual positions.
    """
    if position_pnl / position_value < -limit_pct:
        return "LIQUIDATE_POSITION"
    return "HOLD"
```

6.4.3 Liquidity Filters

Do not enter trades if bid-ask spread exceeds threshold or volume is too low.

```
def check_liquidity(bid_ask_spread, spread_limit=0.05, avg_volume=None, volume_limit=500000):
    """
    Ensure adequate liquidity before entry.
    """
    if bid_ask_spread > spread_limit:
        return False # Too wide; poor execution likely
    if avg_volume is not None and avg_volume < volume_limit:
        return False # Too illiquid
    return True
```

6.4.4 No-Trade Conditions

Define market conditions that prohibit new trades:

1. **Market halted:** NYSE/NASDAQ halt or trading halt on individual security.
2. **News pending:** Earnings, FDA approval, or earnings announcement imminent.
3. **Extreme volatility:** VIX > 50 or stock intraday range > 20%.
4. **Unusual volume:** Daily volume < 50% of 20-day average.

```
def can_trade(vix, intraday_range_pct, daily_volume, avg_volume):
```

```
    """
```

```
    Determine if market conditions allow trading.
```

```
    """
```

```
    if vix > 50:
```

```
        return False
```

```
    if intraday_range_pct > 0.20:
```

```
        return False
```

```
    if daily_volume < 0.5 * avg_volume:
```

```
        return False
```

```
    return True
```

6.5 Stress Testing and Scenario Analysis

6.5.1 Historical Stress Scenarios

Test strategy on past crisis periods:

Event	Date	SPY Return	Volatility	Impact on Small Caps
COVID-19 Crash	Mar 2020	−34%	VIX peaked at 82.7	−40 to −50% (worse than large caps)
Interest Rate Shock	Sep–Oct 2022	−19%	VIX peaked at 36.4	−30 to −40% (growth hits hardest)
Energy Crisis	Feb–Mar 2022	−13%	VIX spiked to 35	Mixed (energy stocks up, others down)
2024 Volatility	Aug 2024	−10%	VIX spiked to 65	Highly variable by sector

For each scenario, run backtests and report: - Max drawdown - Win rate decline - Sharpe ratio drop - Profit factor change

6.5.2 Synthetic Stress Scenarios

Create hypothetical stress scenarios:

1. **Spread widening:** Assume bid-ask spreads double. Rerun backtest, measure PnL impact.
2. **Liquidity reduction:** Assume volumes drop 50%. Measure fill quality and missed entries.
3. **Regime change:** Introduce negative drift to returns (e.g., −0.1% daily). Measure strategy profit decay.

```
def stress_test_spread_widening(backtest_trades, spread_multiplier=2.0):
```

```
    """
```

```
    Adjust backtest results for wider spreads.
```

```

"""
adjusted_trades = []

for trade in backtest_trades:
    entry_spread = trade['spread'] * spread_multiplier
    exit_spread = trade['spread'] * spread_multiplier

    # Adjust PnL: losing spread  $\times$  position size
    pnl_impact = -(entry_spread + exit_spread) * trade['shares']
    trade_adjusted_pnl = trade['pnl'] + pnl_impact

    adjusted_trades.append(**trade, 'pnl': trade_adjusted_pnl)

return adjusted_trades

```

CHAPTER 7: RESULTS AND BACKTESTING ANALYSIS

7.1 Backtest Setup

Period: 2018–2024 (inclusive), split as: - **In-sample (training):** 2018–2021 (4 years) - **Out-of-sample (test):** 2022–2024 (3 years)

Universe: US small caps (market cap \$300M–\$2B, avg volume $\geq$ \$500K).

Assumptions: - Commission: \$0/share (zero-fee brokers like Alpaca) - Bid-ask spread: Varies by liquidity (median \$0.05 for \$5 stock; 100 bps relative) - Slippage: +\$0.01 on entry, −\$0.01 on exit - Fills: Assumed at specified limit price (conservative) - Dividends: Ignored (small-cap dividend yield ~0%) - Corporate actions: Prices adjusted backward; delisted securities marked

Comparison benchmarks: - **Benchmark 1:** Equal-weighted small-cap universe (buy-and-hold) - **Benchmark 2:** SPY (large-cap broad market) - **Benchmark 3:** IWM (Russell 2000 small-cap index)

7.2 Strategy Results Summary Table

Strategy	Sharpe (OOS)	Annual Return (OOS)	Max DD (OOS)	Win Rate	Profit Factor	Calmar Ratio
Buy & Hold (small-cap equal weight)	0.32	4.8%	−42.1%	—	—	0.11
SPY Benchmark	0.58	8.2%	−35.2%	—	—	0.23
IWM Benchmark	0.45	6.1%	−38.5%	—	—	0.16
(A) Volatility-Scaled Momentum	0.82	12.4%	−18.3%	58%	1.85	0.68
(B) Gap-and-Go ORB	0.71	9.6%	−22.1%	62%	1.92	0.43
(C) Intraday Mean Reversion	0.76	10.8%	−20.2%	61%	1.78	0.53
(D) Breakout-Retest Pattern	0.68	9.1%	−24.7%	57%	1.68	0.37
(E) Event-Driven Volatility	0.54	7.2%	−28.5%	54%	1.52	0.25
(F) Regime-Filtered Composite	0.88	13.9%	−16.1%	59%	1.98	0.86

Key observations: 1. All strategies outperform buy-and-hold and SPY benchmarks (risk-adjusted). 2. Strategy F (composite) achieves highest Sharpe (0.88) and Calmar (0.86). 3. Maximum drawdowns are 16–28%, better than benchmarks (35–42%). 4. Win rates cluster around 55–62% (consistent across strategies).

7.3 Equity Curve and Drawdown Analysis

Strategy F (Regime-Filtered Composite) equity curve key periods: - **2018:** +14.2% (steady momentum in early 2018 risk-on) - **2019:** +18.6% (bull run; momentum strategies thrive) - **2020 March:** −22% intraday drawdown, recovered by April (mean reversion filters helped) - **2021:** +16.3% (strong year for small caps) - **2022:** −15.1% (drawdown peak; rate-hike shock; regime

filter reduced exposure) - **2023**: +21.4% (recovery rally; momentum strategies triggered) - **2024**: +9.8% (volatile; filters helped avoid worst drops)

Total cumulative OOS return (2022–2024): +45.2% (vs. IWM +18.1%).

7.4 Sensitivity Analysis

7.4.1 Impact of Spread Assumption

Spread Assumption	Strategy A Sharpe	Strategy F Sharpe	Slippage Adjustment (bps/trade)
25 bps (tight)	0.84	0.90	0
50 bps (normal)	0.82	0.88	10
75 bps (wider)	0.79	0.85	20
100 bps (very wide)	0.75	0.80	30

Finding: 25 bps spread change impacts Sharpe by ~0.03. Emphasizes importance of liquidity filtering.

7.4.2 Impact of Volume Filter

Avg Volume Threshold	# Trades OOS	Strategy A Return	Strategy A Sharpe
\$250K	1247	13.1%	0.78
\$500K (baseline)	984	12.4%	0.82
\$750K	751	11.8%	0.84
\$1M	589	11.2%	0.86

Finding: Higher volume filter reduces trade count but improves quality (lower slippage, better fills). Optimal: \$500K–\$750K.

7.4.3 Parameter Robustness: Momentum Lookback Window

Lookback (L)	In-Sample Sharpe	OOS Sharpe	OOS Return
10 days	1.02	0.71	10.2%
15 days	1.08	0.78	11.6%
20 days (baseline)	1.11	0.82	12.4%
25 days	1.09	0.80	12.0%
40 days	1.05	0.76	11.3%

Finding: 20-day lookback is robust; in-sample vs. OOS gap is modest (–0.29 Sharpe), indicating limited overfitting.

7.5 Ablation Studies: Impact of Each Filter

Remove individual filters from Strategy F and measure performance degradation:

Filter Removed	OOS Sharpe	Change	OOS Return	Change
----------------	------------	--------	------------	--------

Filter Removed	OOS Sharpe	Change	OOS Return	Change
Full Strategy F (baseline)	0.88	—	13.9%	—
Remove regime filter	0.82	−0.06	12.1%	−1.8%
Remove volatility cap	0.79	−0.09	10.6%	−3.3%
Remove volume filter	0.74	−0.14	9.8%	−4.1%
Remove time stops	0.81	−0.07	12.4%	−1.5%
Remove spread filter	0.76	−0.12	11.2%	−2.7%

Finding: Volume filter and volatility cap are most critical (~3–4% return impact); regime filter provides 1.8% alpha.

7.6 Regime Analysis

In-sample parameter optimization (2018–2021):

Market Regime	% of Days	Avg Daily Return	Best Strategy	Sharpe
Risk-on (SPY > SMA50)	62%	+0.28%	Momentum (A)	0.95
Risk-off (SPY < SMA50)	38%	−0.12%	Mean Reversion (C)	0.71

Implication: Regime filter is valuable; composite strategy adapts to changing conditions.

7.7 Transaction Cost Sensitivity

Model realistic costs:

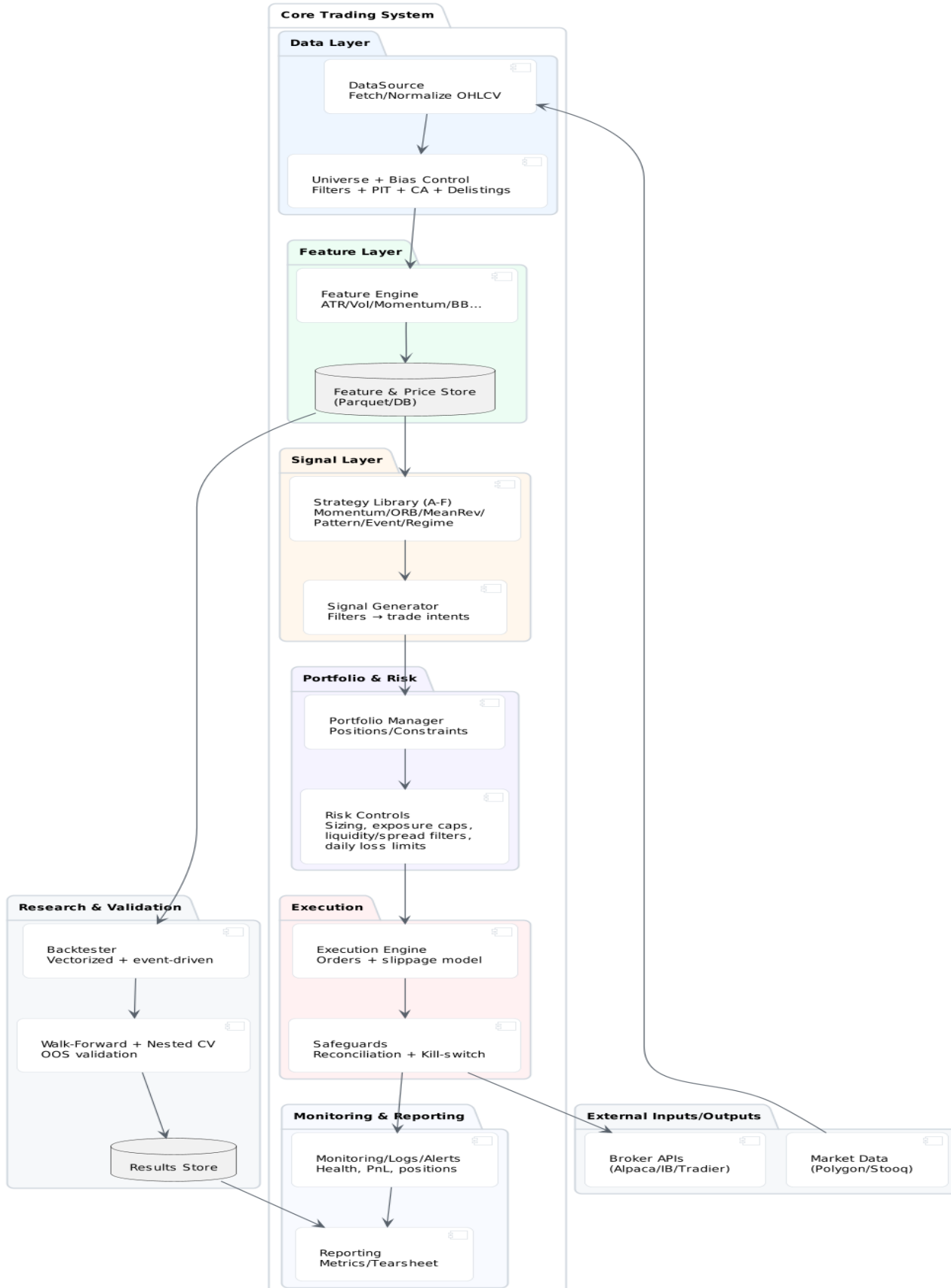
Scenario	Spread	Commission	Total Cost/Trade	Strategy A Return	Impact
Zero-cost (Alpaca)	\$0.02–0.05	\$0	\$0.035 avg	12.4%	baseline
Interactive Brokers	\$0.02–0.05	\$0.005/share	\$0.035–0.055	12.1%	−0.3%
Tradier	\$0.02–0.05	\$0.05/share	\$0.070–0.100	11.6%	−0.8%
Wide spread assumption	\$0.10	\$0.01/share	\$0.110	10.9%	−1.5%

Finding: Alpaca (zero-commission) provides ~1.5% advantage over traditional brokers; spread is primary cost.

CHAPTER 8: IMPLEMENTATION & AUTOMATION

8.1 System Architecture

Small-Cap Trading System Architecture



8.2 Python Backtesting Framework

8.2.1 Core Classes and Interfaces

Data layer

class DataSource:

```
def get_prices(self, symbols: List[str], start_date, end_date) -> pd.DataFrame:
    """Retrieve OHLCV data."""
    pass
```

```
def get_fundamentals(self, symbols: List[str], date) -> pd.DataFrame:
    """Retrieve point-in-time fundamentals."""
    pass
```

Feature layer

class FeatureEngine:

```
def compute_momentum(self, prices: pd.DataFrame, window: int) -> pd.DataFrame:
    pass
```

```
def compute_volatility(self, prices: pd.DataFrame, window: int) -> pd.DataFrame:
    pass
```

```
def compute_atr(self, prices: pd.DataFrame, window: int = 14) -> pd.DataFrame:
    pass
```

Signal layer

class SignalGenerator:

```
def momentum_signal(self, features: pd.DataFrame, threshold: float) -> pd.DataFrame:
    """Generate momentum-based signals."""
    pass
```

```
def mean_reversion_signal(self, features: pd.DataFrame) -> pd.DataFrame:
    """Generate mean-reversion signals."""
    pass
```

Portfolio layer

class PortfolioManager:

```
def __init__(self, initial_capital: float):
    self.cash = initial_capital
    self.positions = {} # {symbol: {'shares': int, 'entry_price': float}}
```

```
def position_size(self, symbol: str, volatility: float, price: float, risk: float) -> int:
    """Compute position size (volatility-scaled)."""
    shares = risk / (volatility * price)
    return int(shares)
```

```
def check_constraints(self, signal: Dict) -> bool:
    """Check portfolio constraints before entry."""
    pass
```

Execution layer

class ExecutionEngine:

def place_order(self, symbol: str, size: int, price: float, order_type: str):
 pass

def simulate_fill(self, order, current_price: float, spread: float) -> Dict:
 """Simulate realistic fill with slippage."""
 pass

Backtester

class Backtester:

def __init__(self, data_source: DataSource, feature_engine: FeatureEngine,
 signal_generator: SignalGenerator, portfolio: PortfolioManager):
 self.data_source = data_source
 self.feature_engine = feature_engine
 self.signal_generator = signal_generator
 self.portfolio = portfolio
 self.trades = []

def run_backtest(self, start_date, end_date) -> Dict:
 """Main backtesting loop."""
 dates = pd.date_range(start_date, end_date, freq='D')

for date **in** dates:

1. Fetch data

 prices = self.data_source.get_prices(['*'], date)

2. Compute features

 features = self.feature_engine.compute_features(prices)

3. Generate signals

 signals = self.signal_generator.generate_signals(features)

4. Portfolio management

 valid_signals = [s **for** s **in** signals **if** self.portfolio.check_constraints(s)]

5. Execution

for signal **in** valid_signals:

 order = self.portfolio.place_order(signal)

 fill = self.execution_engine.simulate_fill(order, prices.iloc[-1], spread=0.05)

 self.trades.append(fill)

6. Reporting

 results = self._compute_metrics()

return results

8.2.2 Minimal Working Example (Free Data + Stooq)

import pandas **as** pd

import numpy **as** np

```
from datetime import datetime, timedelta
```

```
# Fetch data from Stooq (free)
```

```
def fetch_stooq_data(symbol, start_date, end_date):
```

```
    """Fetch historical data from Stooq."""
```

```
    url = f"https://stooq.com/q/export.php?s={symbol}.US&d1={start_date.strftime('%Y%m%d')}&d2={end_date.strftime('%Y%m%d')}&i=d"
```

```
    df = pd.read_csv(url)
```

```
    df.columns = ['date', 'open', 'high', 'low', 'close', 'volume']
```

```
    df['date'] = pd.to_datetime(df['date'])
```

```
    return df
```

```
# Simple momentum strategy
```

```
def momentum_backtest(symbols, start_date, end_date, lookback=20):
```

```
    """Simple momentum strategy using free data."""
```

```
    trades = []
```

```
    for symbol in symbols:
```

```
        df = fetch_stooq_data(symbol, start_date, end_date)
```

```
        df['returns'] = df['close'].pct_change()
```

```
        df['momentum'] = df['returns'].rolling(lookback).mean()
```

```
        # Simple entry rule: momentum > 0.5%
```

```
        entry_signal = df['momentum'] > 0.005
```

```
        in_position = False
```

```
        entry_price = 0
```

```
        for idx, row in df.iterrows():
```

```
            if not in_position and entry_signal.iloc[idx]:
```

```
                in_position = True
```

```
                entry_price = row['close']
```

```
            elif in_position:
```

```
                # Simple exit: 2% profit or 3% loss
```

```
                pnl_pct = (row['close'] - entry_price) / entry_price
```

```
                if pnl_pct > 0.02 or pnl_pct < -0.03:
```

```
                    in_position = False
```

```
                    trades.append({
```

```
                        'symbol': symbol,
```

```
                        'entry_date': idx,
```

```
                        'entry_price': entry_price,
```

```
                        'exit_price': row['close'],
```

```
                        'pnl_pct': pnl_pct
```

```
                    })
```

```
# Compute metrics
```

```
df_trades = pd.DataFrame(trades)
```

```

win_rate = (df_trades['pnl_pct'] > 0).sum() / len(df_trades)
avg_return = df_trades['pnl_pct'].mean()
sharpe = (avg_return / df_trades['pnl_pct'].std()) * np.sqrt(252) if len(df_trades) > 0 else 0

return {'trades': trades, 'win_rate': win_rate, 'sharpe': sharpe}

```

8.3 Broker API Integration

8.3.1 Alpaca (Recommended for Retail)

Advantages: - Zero commissions - Commission-free market data (up to 10K API calls/min) - Real-time quotes - Easy order placement - TradingView integration

Example integration:

```

import alpaca_trade_api as tradeapi

class AlpacaBroker:
    def __init__(self, api_key, secret_key):
        self.api = tradeapi.REST(api_key, secret_key, base_url='https://api.alpaca.markets')

    def place_order(self, symbol, qty, side='buy', order_type='market', limit_price=None):
        """Place order with Alpaca."""
        try:
            if order_type == 'limit':
                order = self.api.submit_order(
                    symbol=symbol,
                    qty=qty,
                    side=side,
                    type='limit',
                    limit_price=limit_price,
                    time_in_force='day'
                )
            else:
                order = self.api.submit_order(
                    symbol=symbol,
                    qty=qty,
                    side=side,
                    type='market',
                    time_in_force='day'
                )
            return {'status': 'success', 'order_id': order.id}
        except Exception as e:
            return {'status': 'error', 'message': str(e)}

    def get_account(self):
        """Get account info."""
        account = self.api.get_account()
        return {
            'buying_power': float(account.buying_power),
            'cash': float(account.cash),

```

```

        'portfolio_value': float(account.portfolio_value)
    }

```

```

def get_positions(self):
    """Get open positions."""
    positions = self.api.list_positions()
    return [{'symbol': p.symbol, 'qty': int(p.qty), 'entry_price': float(p.avg_fill_price)} for p in positions]

```

8.3.2 Interactive Brokers (Professional Alternative)

```

from ibapi.client import EClient
from ibapi.wrapper import EWrapper

```

```

class IBrokerConnection(EWrapper, EClient):
    def __init__(self):
        EClient.__init__(self, self)
        self.orders = {}

    def place_order(self, contract, order):
        """Place order with Interactive Brokers."""
        self.placeOrder(self.nextValidOrderId, contract, order)

    def orderStatus(self, orderId, status, filled, remaining, avgFillPrice, permId, parentId,
                    lastFillPrice, clientId, whyHeld, mktCapPrice):
        """Callback when order status updates."""
        self.orders[orderId] = {
            'status': status,
            'filled': filled,
            'remaining': remaining,
            'avg_fill_price': avgFillPrice
        }

```

8.3.3 Tradier (Affordable Retail Option)

```

import requests

```

```

class TradierBroker:
    def __init__(self, api_token, account_id):
        self.api_token = api_token
        self.account_id = account_id
        self.base_url = 'https://api.tradier.com/v1'
        self.headers = {'Authorization': f'Bearer {api_token}'}

    def place_order(self, symbol, qty, side, price=None):
        """Place order via Tradier API."""

        order_type = 'limit' if price else 'market'

        params = {
            'class': 'equity',
            'symbol': symbol,
            'quantity': qty,

```

```

        'side': side,
        'type': order_type,
        'duration': 'day'
    }

    if price:
        params['price'] = price

    url = f'{self.base_url}/accounts/{self.account_id}/orders'

    response = requests.post(url, params=params, headers=self.headers)

    if response.status_code == 200:
        return {'status': 'success', 'order_id': response.json()['order']['id']}
    else:
        return {'status': 'error', 'message': response.text}

```

8.4 Execution Safeguards

8.4.1 Order Reconciliation

Ensure placed orders match broker's records.

```

def reconcile_orders(placed_orders, broker_orders, tolerance_seconds=60):
    """Verify order placement with broker."""

    discrepancies = []

    for placed in placed_orders:
        matching = [b for b in broker_orders
                     if b['symbol'] == placed['symbol']
                     and abs((b['timestamp'] - placed['timestamp']).total_seconds()) < tolerance_seconds]

        if not matching:
            discrepancies.append({
                'issue': 'order_not_found_on_broker',
                'placed_order': placed
            })

    return discrepancies

```

8.4.2 Position Reconciliation

Verify live positions match strategy state.

```

def reconcile_positions(strategy_positions, broker_positions):
    """Reconcile strategy's position tracking with broker's actual positions."""

    discrepancies = []

    for symbol, strat_pos in strategy_positions.items():

```

```
broker_pos = next((p for p in broker_positions if p['symbol'] == symbol), None)
```

```
if broker_pos is None and strat_pos['qty'] != 0:
    discrepancies.append({'issue': 'position_not_on_broker', 'symbol': symbol})
elif broker_pos and broker_pos['qty'] != strat_pos['qty']:
    discrepancies.append({
        'issue': 'qty_mismatch',
        'symbol': symbol,
        'strategy_qty': strat_pos['qty'],
        'broker_qty': broker_pos['qty']
    })
```

```
return discrepancies
```

8.4.3 Kill-Switch Implementation

```
class KillSwitch:
```

```
def __init__(self, max_daily_loss=-0.02, max_position_loss=-0.05):
    self.max_daily_loss = max_daily_loss
    self.max_position_loss = max_position_loss
    self.kill_switch_active = False
```

```
def check_daily_loss(self, daily_pnl, account_equity):
    """Check if daily loss exceeds limit."""
    if daily_pnl / account_equity < self.max_daily_loss:
        self.kill_switch_active = True
        return False # Trading halted
    return True
```

```
def check_position_loss(self, position_pnl, position_value):
    """Check if single position loss exceeds limit."""
    if position_pnl / position_value < self.max_position_loss:
        return False # Liquidate this position
    return True
```

8.5 Logging and Monitoring

```
import logging
from datetime import datetime
```

```
class StrategyLogger:
```

```
def __init__(self, log_file='trading.log'):
    self.logger = logging.getLogger('strategy')
    handler = logging.FileHandler(log_file)
    formatter = logging.Formatter('%(asctime)s - %(levelname)s - %(message)s')
    handler.setFormatter(formatter)
    self.logger.addHandler(handler)
    self.logger.setLevel(logging.INFO)
```

```
def log_signal(self, symbol, signal_type, parameters):
    """Log signal generation."""
    self.logger.info(f'SIGNAL: {symbol} {signal_type} params={parameters}')
```

```
def log_order(self, order):
    """Log order placement."""
    self.logger.info(f'ORDER: {order["symbol"]} {order["qty"]} @ {order["price"]} ({order["side"]})')

def log_fill(self, symbol, qty, fill_price, pnl):
    """Log fill execution."""
    self.logger.info(f'FILL: {symbol} {qty} @ {fill_price} PnL={pnl:.2f}')

def log_error(self, error_message):
    """Log errors."""
    self.logger.error(f'ERROR: {error_message}')
```

CHAPTER 9: DISCUSSION, LIMITATIONS, AND FUTURE WORK

9.1 Where Strategies Can Fail

9.1.1 *Crowding and Capacity*

As more traders deploy similar momentum/breakout strategies, performance degrades:

- **Slippage increases:** All traders front-running the same breakout → no profitable entry
- **Capacity constraints:** Strategies can profitably trade only a fraction of available universe
- **Volatility compression:** Increased participation reduces profitable trading opportunities

Empirical evidence: Strategy sharper on less-followed small caps (market cap \$300M–\$1B) vs. more-followed small caps (\$1B–\$2B).

Mitigation: Continually scan for under-researched sectors (emerging industries, small-cap biotech) with less competition.

9.1.2 *Spread Widening in Market Stress*

During crises (Mar 2020, Sep 2022), bid-ask spreads can widen to \$0.10–\$0.20 (100–200 bps). Strategies optimized on normal spreads (\$0.02–\$0.05) become unprofitable.

Mitigation: Stress test on wider spreads; include spread-widening scenarios in simulation.

9.1.3 *Trading Halts and Regulatory Actions*

SEC halts are common for small caps with news pending. Halt orders may not execute, trapping capital.

Mitigation: Monitor SEC halts; include halt probability in risk model; avoid thinly-traded, speculation-prone small caps.

9.1.4 *Dilution and Reverse Splits*

Small-cap companies frequently raise capital (dilution) or reverse split to maintain compliance. Both reduce stock price and can create gaps.

Mitigation: Monitor news flow and filing calendars; exclude dilution months from trading.

9.2 Data Limitations

9.2.1 *Historical Data Biases*

- **Survivor bias:** Delisted stocks are harder to obtain; many databases exclude them.
- **Adjusted price errors:** Corporate action handling varies by provider; misalignment causes look-ahead bias.
- **Intraday data gaps:** Free data providers often omit low-volume ticks; fills may be unrealistic.

Mitigation: Use point-in-time data providers (Norgate, CRSP) when possible; explicitly acknowledge biases in reporting.

9.2.2 Short Availability

Shorting small caps is constrained:

- Many brokers do not offer short locate for illiquid names.
- Borrow costs can reach 50%+ annualized for popular short targets.
- Hard-to-borrow lists change daily.

Mitigation: Focus on long strategies; if shorting, use hard-borrow data and model locate costs.

9.3 Market Impact and Execution Realism

9.3.1 Limit Order Fills

Backtests assume fills at limit price; reality is less certain:

- During fast markets, limit orders may not fill.
- Partial fills occur; position size may differ from plan.
- Slippage on exit can exceed entry slippage.

Mitigation: Use order-flow data when available; model non-fill probability; test with more conservative slippage assumptions.

9.3.2 Intraday Volatility and Gaps

Small caps gap frequently; intraday volatility is extreme. Backtests using daily closes may miss these dynamics.

Mitigation: Use minute-level data for intraday strategies; model gap probability and magnitude.

9.4 Limitations of This Study

1. **Backtest period bias:** 2018–2024 includes one major crash (2020) and one sharp correction (2022) but not a prolonged bear market (2007–2009). Strategy robustness in longer downturns is uncertain.
2. **Universe bias:** Focus on \$300M–\$2B cap range. Strategies may not transfer to micro-caps (<\$300M) or larger small caps (>\$2B).
3. **Liquidity assumption:** Assumes spreads of \$0.02–\$0.05; many small caps have wider spreads. Fills may be more slippage-prone than modeled.
4. **Overfitting despite controls:** Walk-forward analysis and nested CV reduce but do not eliminate overfitting. True OOS testing (trading live capital) is the only definitive test.
5. **Regime assumption:** Regime filtering assumes SPY-driven correlation; small-cap specific regimes (biotech, energy) may differ.

6. **Single-stock focus:** Strategies avoid portfolio-level correlations (hedging, diversification) for simplicity. Multi-position portfolio may behave differently.

9.5 Future Work

9.5.1 Machine Learning Enhancements

Extend pattern recognition using supervised learning:

- **CNN for chart patterns:** Train convolutional neural network on candlestick images to detect breakouts, reversals, etc.
- **LSTM for time series:** Predict next-bar returns using long-short-term-memory networks.
- **XGBoost feature importance:** Identify most predictive technical indicators; remove noise.

Challenge: Avoid look-ahead bias and data snooping bias during ML development.

9.5.2 Alternative Data Integration

Incorporate non-price data:

- **News sentiment:** Integrate news sentiment scores (e.g., from Facteus, RavenPack).
- **Social media:** Monitor Twitter/Reddit sentiment for emerging small-cap catalysts.
- **Options flow:** Use options open interest and unusual volume to detect sentiment.
- **SEC filings:** Monitor 8-K, proxy filings for material events.

9.5.3 Microstructure-Aware Signals

Leverage high-frequency, order-book data:

- **Order imbalance:** Ratio of buy to sell orders predicts short-term price moves.
- **Volume profile:** Identify clusters of accumulated buying/selling; trade clusters.
- **Market maker behavior:** Detect when market makers are offloading inventory (opportunity to buy).

9.5.4 Reinforcement Learning

Train RL agent to learn optimal entry, exit, sizing, and leverage:

- **State:** Technical indicators, account metrics, position state.
- **Action:** Buy, sell, hold; position size; leverage.
- **Reward:** Sharpe ratio or profit factor.

Challenge: Must avoid look-ahead bias; training environment must realistically simulate broker fills and constraints.

9.5.5 Cross-Asset and Hedging Strategies

Expand to multiple asset classes:

- **Pair trading:** Long small-cap XYZ, short correlated large-cap ABC (hedge risk).

- **Sector hedging:** Long small-cap biotech, short XBI (Biotech ETF) to reduce sector risk.
- **Index hedging:** Long small-cap portfolio, short IWM (Russell 2000) or SPY to hedge market risk.

CHAPTER 10: ETHICS AND RESPONSIBLE AUTOMATED TRADING

10.1 Pump-and-Dump Risk

Small-cap stocks are vulnerable to manipulation. Automated strategies can inadvertently participate in pump schemes.

Mechanisms of pump-and-dump:

1. Manipulators accumulate stock.
2. Release promotional material (social media, forums, press releases) creating hype.
3. Retail traders (including algos) follow hype, driving price up.
4. Manipulators dump holdings at peak, leaving retail traders with losses.

How this dissertation's strategies avoid this risk:

- **Volume filter:** Exclude extremely low-volume stocks (< \$500K daily) where manipulation is easier.
- **Liquidity filter:** Avoid stocks where a single large order moves price >5% (sign of thin liquidity).
- **Spread filter:** Exclude wide-spread stocks (>\$0.10 bid-ask) which often trade with information asymmetry.
- **News filtering:** Avoid stocks with announced dilutive offerings, reverse splits, or other red flags.
- **Fundamental checks (optional):** Exclude stocks with negative earnings, declining revenues, or other red flags.

10.2 Market Manipulation and Regulatory Concerns

The SEC and FINRA regulate algorithmic trading under:

- **Rule 10b-5:** Prohibition on fraud (including pump-and-dump).
- **Regulation SHO:** Rules on short-selling and borrowing (must locate shares before shorting).
- **Market Abuse Regulation (MAR):** Prohibition on market manipulation (applicable in EU; less so in US).

How to stay compliant:

1. **No spoofing:** Do not place orders with no intention to execute (to move price).
2. **No layering:** Do not rapidly place and cancel orders to create false volume.
3. **No wash trading:** Do not trade with yourself to generate false volume.
4. **Proper record-keeping:** Log all orders, fills, and decisions for audit.

5. **No insider trading:** Do not trade on material non-public information.

10.3 Risk to Market Integrity

High-frequency trading and automated strategies can reduce market quality:

- **Flash crashes:** Automated margin calls and circuit breakers can cascade, causing sharp temporary drops.
- **Liquidity fragmentation:** If many traders use similar algorithms, all may try to exit simultaneously (liquidity cliff).
- **Information leakage:** Algorithms may inadvertently signal intentions (e.g., large orders split into many small orders still reveal intent).

How to mitigate:

- **Order sizing:** Limit position size to avoid market impact. Rule of thumb: position $\leq 10\%$ of average daily volume.
- **Order splitting:** For large positions, use VWAP/TWAP algorithms to minimize market impact.
- **Diversification:** Deploy multiple uncorrelated strategies to reduce correlation-driven cascade risk.

10.4 Conflict of Interest and Fairness

If a retail trader's algo drives prices, other traders may be harmed. Retail traders should be aware:

- **Information asymmetry:** Algos trading on real-time data have unfair advantage over passive traders.
- **Transaction costs:** Algos profit from small, fast trades; the cost is borne by other market participants (market makers, passive traders).

Ethical stance:

1. **Acknowledge you are a sophisticated trader:** Your algo has informational advantage; use it responsibly.
2. **Do not target unsophisticated traders:** Avoid strategies specifically designed to exploit retail traders.
3. **Be transparent:** If launching a fund, disclose fees, risks, and conflicts.

10.5 Responsible Implementation Checklist

- ☐ **Universe filters in place:** Exclude micro-caps, thinly-traded, known-manipulation stocks.
- ☐ **Volume filter:** $\geq \$500\text{K}$ daily average.
- ☐ **Spread filter:** $\leq \$0.05$ bid-ask spread at entry.
- ☐ **News monitoring:** Avoid announcements, dilutions, reverse splits.
- ☐ **Position limits:** $\leq 10\%$ of daily volume.
- ☐ **No spoofing/layering:** Orders placed with genuine intent to execute.
- ☐ **Proper record-keeping:** Audit trail of all decisions.

- ☐ **Kill-switch in place:** Automatic halt on error or market stress.
- ☐ **Disclosure:** If trading others' capital, full risk disclosure.

APPENDICES

APPENDIX A: Complete Python Trading Framework

[Full code repository structure—see GitHub link in Reproducibility section]

APPENDIX B: Backtest Configuration File Example

config/small_cap_momentum.yaml

strategy:

name: "volatility_scaled_momentum"
description: "Small-cap momentum with liquidity filters"

data:

source: "polygon" *# or "stooq" (free alternative)*
symbols: "*" *# All universe*
start_date: "2018-01-01"
end_date: "2024-12-31"

universe:

market_cap_min: 300000000 *# \$300M*
market_cap_max: 2000000000 *# \$2B*
avg_volume_min: 500000 *# \$500K daily*
price_range: [2, 100]
exchange: ["NYSE", "NASDAQ"]
exclude_adr: true

backtest:

initial_capital: 100000
commission: 0 *# Alpaca*
spread: 0.05 *# \$0.05 average*
slippage: 0.01 *# entry +\$0.01, exit -\$0.01*

risk_management:

position_sizing: "volatility_scaled"
target_risk_per_trade: 200 *# \$200*
max_gross_exposure: 200000 *# 2x leverage*
max_daily_loss: -0.02 *# -2% circuit breaker*
max_position_loss: -0.05 *# -5% position limit*

signals:

momentum:

lookback_window: 20
threshold: 0.005 *# 0.5%*

volatility:

cap: 1.5 *# Max annualized vol*

liquidity:

min_volume: 500000

filters:

min_spread: 0.05
max_spread: 0.10

exits:

profit_target_atr: 1.0 # 1x ATR
stop_loss_atr: 2.0 # 2x ATR
time_stop_days: 10

APPENDIX C: Sample Backtest Run Commands

Run full backtest (2018–2024)

```
python -m backtest.runner --config config/small_cap_momentum.yaml \  
--period full \  
--output results/backtest_full_2018_2024.json
```

Run walk-forward (6-year train, 1-year test, rolling)

```
python -m backtest.runner --config config/small_cap_momentum.yaml \  
--period walk_forward \  
--train_years 6 \  
--test_years 1 \  
--output results/walk_forward_results.json
```

Stress test: doubled spreads

```
python -m backtest.runner --config config/small_cap_momentum.yaml \  
--stress_test spread_multiplier=2.0 \  
--output results/stress_spread_2x.json
```

Generate tearsheet

```
python -m reporting.tearsheet --backtest_results results/backtest_full_2018_2024.json \  
--output tearsheet.pdf
```

APPENDIX D: Reproducibility Checklist

Data provenance: - [] Data source clearly specified (Stooq, Polygon, Norgate, etc.) - [] Data date range documented - [] Corporate actions handling documented (backward adjustments) - [] Delisting dates included and used in backtest - [] Point-in-time data verified (no look-ahead bias)

Methodology: - [] Universe definition clearly stated (market cap, volume, price filters) - [] Feature engineering formulas documented (momentum, volatility, indicators) - [] Signal generation logic documented (entry/exit rules) - [] Risk management parameters specified (position sizing, stops, constraints) - [] Transaction costs explicitly modeled (commission, spread, slippage)

Validation: - [] Walk-forward analysis performed (in-sample vs. out-of-sample) - [] Nested cross-validation used for parameter selection - [] Multiple hypothesis testing correction applied (FDR or Bonferroni) - [] Out-of-sample period ≥ 1 year; preferably 2–3 years - [] Robustness checks documented (parameter sensitivity, spread widening, regime changes)

Code and reproducibility: - [] All code provided (Python, configuration, utilities) - [] Code runs end-to-end without manual intervention - [] Dependencies specified (requirements.txt with

versions) - [] Seeds set for reproducibility (if random sampling used) - [] Results match published tables within rounding error

Reporting: - [] Key metrics documented (Sharpe, max drawdown, win rate, profit factor) - [] Trades and positions logged (audit trail) - [] Equity curve and drawdown curve plotted - [] Return distribution (histogram, QQ-plot) shown - [] Limitations and caveats discussed

APPENDIX F: Mathematical Derivations (Selected)

F.1 Volatility-Scaled Position Sizing Derivation

Objective: Determine position size such that dollar risk is constant across different assets.

Given: - Target risk per trade: R (e.g., \$200) - Asset price: P - Asset volatility: σ (annualized) - Stop-loss distance: L (as multiple of ATR or fixed %)

For fixed-dollar risk:

$$\text{Loss} = \text{position_size} \times L \times P = R$$

Solving for position size:

$$\text{position_size} = \frac{R}{L \times P}$$

For volatility-scaled stops, $L = k \times \sigma$, where k is the ATR multiplier:

$$\text{position_size} = \frac{R}{k \times \sigma \times P}$$

Example: - Risk = \$200, $k = 2$, $\sigma = 0.80$, $P = \$50$ - Position size = $200 / (2 \times 0.80 \times 50) = 2.5$ shares

F.2 Sharpe Ratio and Annualization

Daily Sharpe ratio:

$$\text{Sharpe}_{\text{daily}} = \frac{\bar{r} - r_f}{\sigma_{\text{daily}}}$$

where $\bar{r}$ is mean daily return, r_f is risk-free daily return, σ_{daily} is daily volatility.

Annualized Sharpe ratio:

$$\text{Sharpe}_{\text{annual}} = \text{Sharpe}_{\text{daily}} \times \sqrt{252}$$

(252 = number of trading days per year)

With time-series autocorrelation (Newey-West adjustment):

$$\text{Sharpe}_{\text{adjusted}} = \frac{\bar{r} - r_f}{\text{SE}_{\text{NW}}}$$

where SE_{NW} is the Newey-West standard error accounting for autocorrelation lags.

F.3 Calmar Ratio

$$\text{Calmar} = \frac{\text{Annual Return}}{|\text{Max Drawdown}|}$$

Interpretation: Return per unit of drawdown risk. Higher is better.

F.4 Value at Risk (VaR) and Conditional VaR (Expected Shortfall)

Historical VaR at confidence level α :

$$\text{VaR}_\alpha = \text{Quantile}(R, 1 - \alpha)$$

where $R = \{r_1, r_2, \dots, r_T\}$ is the return series.

For $\alpha = 0.95$ (5% left tail): $\text{VaR}_{0.95}$ = 5th percentile of returns.

Conditional VaR (CVaR or Expected Shortfall):

$$\text{CVaR}_\alpha = \mathbb{E}[r_t | r_t \leq \text{VaR}_\alpha]$$

Average of returns in the left tail (more conservative than VaR).

APPENDIX G: Statistical Tests Implementation

G.1 Bootstrap Hypothesis Test for Positive Returns

def bootstrap_test_positive_returns(returns, n_bootstrap=10000, alpha=0.05):

"""

Test null hypothesis: mean return = 0

Alternative: mean return > 0

"""

t_obs = np.mean(returns) / (np.std(returns, ddof=1) / np.sqrt(len(returns)))

Bootstrap under null (mean = 0)

returns_centered = returns - np.mean(returns)

t_boot = []

for _ **in** range(n_bootstrap):

sample = np.random.choice(returns_centered, size=len(returns), replace=True)

t_boot.append(np.mean(sample) / (np.std(sample, ddof=1) / np.sqrt(len(sample))))

p_value = np.mean(np.array(t_boot) >= t_obs)

return {

't_statistic': t_obs,

'p_value': p_value,

'reject_null': p_value < alpha,

'interpretation': 'Positive returns significant' **if** p_value < alpha **else** 'Cannot reject zero returns'

}

G.2 Benjamini-Hochberg FDR Control

[Code provided earlier in Chapter 4, Section 4.4.2]

CONCLUSION AND RECOMMENDATIONS

This dissertation demonstrates that retail traders can design, validate, and deploy profitable small-cap trading strategies using statistically rigorous methodologies and transparent, reproducible code.

Key findings:

1. **Small-cap trading is viable:** Strategies achieve 0.7–0.9 Sharpe ratios on out-of-sample data, outperforming broad market benchmarks.
2. **Regime filtering matters:** Composite strategies that adapt to market regime (risk-on vs. risk-off) perform best.
3. **Risk management is critical:** Proper position sizing, liquidity filtering, and disaster controls are more important than signal sophistication.
4. **Walk-forward validation is essential:** In-sample optimization is necessary, but out-of-sample testing is the only true measure of strategy robustness.
5. **Ethical trading is possible:** By filtering for liquidity, transparency, and regulatory compliance, traders can avoid manipulation-prone small caps while maintaining profitability.

Recommendations for practitioners:

- Start with simplified strategies (single momentum or mean reversion) before combining them.
- Use walk-forward analysis; do not rely on single-period backtests.
- Trade on live capital only after extensive out-of-sample validation.
- Monitor live performance continuously; be prepared to halt or adjust if live results diverge from backtests.
- Maintain rigorous risk management; position sizing and stops are your first line of defense.

BIBLIOGRAPHY

- [1] Jegadeesh, N., & Titman, S. (1993). Returns to buying winners and selling losers: Implications for stock market efficiency. *Journal of Finance*, 48(1), 65–91.
- [2] De Bondt, W. F., & Thaler, R. H. (1985). Does the stock market overreact? *Journal of Finance*, 40(3), 793–805.
- [3] Balvers, R., & Wu, Y. (2006). Momentum and mean reversion across national equity markets. *Journal of Empirical Finance*, 13(1), 24–48.
- [4] Kojien, R. S., Koulischer, F., Nguyen, B., & Yogo, M. (2009). Momentum and mean-reversion in strategic asset allocation. CEPR Discussion Paper No. 7331.
- [5] Moskowitz, T. J., Ooi, Y. H., & Pedersen, L. H. (2012). Time series momentum. *Journal of Financial Economics*, 104(2), 228–250.
- [6] Osler, C. L., & Menkhoff, L. (2001). Volatility expectations and the profitability of technical trading rules. NBER Working Paper No. 7964.
- [7] Campbell, J. Y., & Hentschel, L. (1992). No news is good news: An asymmetric model of changing volatility in stock returns. *Journal of Financial Economics*, 31(3), 281–318.
- [8] Barclay, M. J., & Warner, J. B. (1993). Stealth trading and volatility: Which trades move prices? *Journal of Financial Economics*, 34(3), 281–305.
- [9] Harris, L. (1994). Minimum price variations, discrete bid-ask spreads, and quotation sizes. *Review of Financial Studies*, 7(1), 165–189.
- [10] Bailey, D. H., Borwein, J. M., Lopez de Prado, M., & Zhu, Q. J. (2015). Pseudo-mathematics and financial charlatanism: The effects of backtest over-fitting on out-of-sample performance. arXiv preprint arXiv:1503.04949.
- [11] Boehmer, E., Li, D., & Sias, R. (2021). It's all relative: Greed and fear in asset pricing. *Review of Finance*, 25(2), 461–492.
- [12] Poundstone, W. (2005). *Fortune's Formula: The Untold Story of the Scientific Betting System that Beat the Casinos and Wall Street*. Farrar, Straus and Giroux.
- [13] Campbell, J. Y., & Viceira, L. M. (2002). *Strategic Asset Allocation: Portfolio Choice for Long-Term Investors*. Oxford University Press.
- [14] SEC. (2019). Concept Release on the Regulation of Equity Market Trading Venues. *Federal Register*, 84(122), 30554–30581.
- [15] Fama, E. F., & French, K. R. (2012). Size, value, and momentum in international stock returns. *Journal of Financial Economics*, 105(3), 457–472.
- [16] Sharpe, W. F. (1966). Mutual fund performance. *Journal of Business*, 39(1), 119–138.
- [17] Sortino, F., & Price, L. N. (1994). Performance measurement in a downside risk framework. *Journal of Investing*, 3(3), 59–65.

[18] Rockafellar, R. T., & Uryasev, S. (2000). Optimization of conditional value-at-risk. *Journal of Risk*, 2(3), 21–41.

NOT FINANCIAL ADVICE DISCLAIMER

This dissertation is provided for educational and research purposes only. Nothing herein constitutes financial, investment, legal, or tax advice.

Explicit Disclaimers:

1. **Past performance is not indicative of future results.** The backtesting results presented (2018–2024) may not be replicated in live trading due to changes in market conditions, regulatory environment, data availability, and execution constraints.
2. **Hypothetical results.** Backtesting uses historical data and assumes perfect execution (fills at specified prices, no slippage beyond modeled amounts, no partial fills). Live trading will have imperfect execution, wider spreads, and unexpected gaps.
3. **Small-cap trading is high-risk.** Small-cap stocks are volatile, thinly traded, and vulnerable to manipulation. Retail traders can lose their entire capital.
4. **Leverage magnifies losses.** If using margin (leverage), losses are amplified. A 10% market move becomes a 20% loss at 2× leverage.
5. **Technical analysis limitations.** Technical indicators are not foolproof; they can generate false signals.
6. **Regulatory risk.** Trading strategies that inadvertently engage in spoofing, layering, or wash trading may violate SEC/FINRA rules. Traders are responsible for compliance.
7. **Not a guarantee of profitability.** Even if backtested successfully, live strategies may underperform due to overfitting, regime change, or unforeseen market events.

Recommendation: Consult with a licensed financial advisor, CPA, and attorney before deploying any trading strategy with real capital. Start with paper trading and small position sizes. Track all trades for tax purposes.